

интеллектуальных указателей ни использовался, вы не должны создавать несколько интеллектуальных указателей для одного обычного указателя.

Пример использования совместно используемых указателей в нескольких контейнерах библиотеки STL описан в разделе 7.11.

Классы `shared_ptr` и `unique_ptr` реализуют разные подходы к работе с массивами и функторами удаления. Класс `unique_ptr` имеет специализацию для массивов, имеющую другой интерфейс. Он более гибкий и меньше влияет на производительность программ, но требует более внимательной работы.

В заключение отметим, что интеллектуальные указатели не являются потокобезопасными, хотя и предоставляют некоторые гарантии. Подробности изложены в разделе 5.2.4.

5.3. Числовые пределы

Числовые типы имеют платформно-зависимые пределы. Стандартная библиотека языка C++ задает эти пределы в шаблоне `numeric_limits`. Эти числовые пределы заменяют и дополняют обычные константы препроцессора языка C, которые по-прежнему доступны для целочисленных типов в заголовках `<climits>` и `<limits.h>` и для чисел с плавающей точкой в заголовках `<cmath>` и `<float.h>`. Новая концепция числовых пределов имеет свои преимущества. Во-первых, она повышает безопасность с точки зрения типов, во-вторых, позволяет программисту писать шаблонные функции, вычисляющие эти пределы.

Числовые пределы обсуждаются в оставшейся части раздела. Однако следует отметить, что всегда лучше писать платформно-независимый код, используя минимальную гарантированную точность для типов. Эти минимальные значения указаны в табл. 5.7.²⁰

Таблица 5.7. Минимальный размер встроенных типов

Тип	Минимальный размер
<code>char</code>	1 байт (8 битов)
<code>short int</code>	2 байта
<code>int</code>	2 байта
<code>long int</code>	4 байта
<code>long long int</code>	8 байтов
<code>float</code>	4 байта
<code>double</code>	8 байтов
<code>long double</code>	8 байтов

Класс `numeric_limits<>`

Обычно шаблоны используются для реализации каких-то операций, которые могут применяться к любому типу. Однако шаблоны также можно использовать для реализации общего интерфейса для каждого типа, что бывает очень полезным. Для этого можно

²⁰ Обратите внимание на то, что здесь байт — это октет из 8 битов. Строго говоря, тип `long int` может состоять из байта, содержащего не менее 32 битов.

предусмотреть специализации общего шаблона. Типичный пример такого приема — класс `numeric_limits`, работающий следующим образом.

- Общий шаблон обеспечивает числовые значения, заданные по умолчанию для любого типа.

```
namespace std {
    // общие числовые пределы, заданные по умолчанию для любого типа
    template <typename T>
    class numeric_limits {
    public:
        // по умолчанию для любого типа T специализация не предусмотрена
        static constexpr bool is_specialized = false;
        ... // другие члены для общего шаблона не нужны
    };
}
```

- Этот общий шаблон для числовых пределов означает, что для типа `T` числовых пределов нет. Для этого член `is_specialized` задается равным `false`.
- Специализации шаблона определяют числовые пределы для каждого числового типа.

```
namespace std {
    // числовые пределы для типа int
    // - определенные реализацией
    template<> class numeric_limits<int> {
    public:
        // да, специализация числовых пределов для типа int существует
        static constexpr bool is_specialized = true;

        static constexpr int min() noexcept {
            return -2147483648;
        }
        static constexpr int max() noexcept {
            return 2147483647;
        }
        static constexpr int digits = 31;
        ...
    };
}
```

Здесь член `is_specialized` установлен равным `true`, а все остальные члены класса имеют значения числовых пределов для конкретного типа.

Общий шаблонный класс `numeric_limits` и его стандартные специализации определены в заголовке `<limits>`. Специализации предусмотрены для любого элементарного типа, который может представлять числовые значения: `bool`, `char`, `signed char`, `unsigned char`, `char16_t`, `char32_t`, `wchar_t`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `float`, `double` и `long double`.²¹ Их легко дополнить пользовательскими числовыми типами.

²¹ Специализации для типов `char16_t`, `char32_t`, `long long` и `unsigned long long` предусмотрены только в стандарте C++11.

Все члены класса `numeric_limits<>` и их описание приведены в табл. 5.8. В ней также указаны соответствующие константы языка C, определенные в заголовках `<climits>`, `<limits.h>`, `<cfloat>` и `<float.h>`.

Таблица 5.8. Члены класса `numeric_limits<>`

Член	Предназначение	Константы языка C
<code>is_specialized</code>	Тип имеет специализацию для числовых пределов	
<code>is_signed</code>	Тип имеет знак	
<code>is_integer</code>	Тип является целочисленным	
<code>is_exact</code>	Вычисления не сопровождаются ошибками округления (<code>true</code> для всех целочисленных типов)	
<code>is_bounded</code>	Набор значений конечен (<code>true</code> для всех встроенных типов)	
<code>is_modulo</code>	Сумма двух положительных чисел может оказаться меньше каждого из них	
<code>is_iec559</code>	Соответствует стандартам IEC 559 и IEEE 754	
<code>min()</code>	Минимальное конечное значение (минимальное нормализованное значение для чисел с плавающей точкой с денормализацией; имеет смысл, если <code>is_bounded</code> и <code>is_signed</code>)	<code>INT_MIN</code> , <code>FLT_MIN</code> , <code>CHAR_MIN</code> , ...
<code>max()</code>	Максимальное конечное число (имеет смысл, если <code>is_bounded</code>)	<code>INT_MAX</code> , <code>FLT_MAX</code> , ...
<code>lowest()</code>	Максимальное отрицательное конечное число (имеет смысл, если <code>is_bounded</code> ; только для стандарта C++11)	
<code>digits</code>	Символ/целое число: количество битов, исключая знаковые (двоичные цифры) Число с плавающей точкой: количество цифр в мантиссе в системе счисления с основанием <code>radix</code>	<code>CHAR_BIT</code> <code>FLT_MANT_DIG</code> , ...
<code>digits10</code>	Количество десятичных знаков (имеет смысл, если <code>is_bounded</code>)	<code>FLT_DIG</code> , ...
<code>max_digits10</code>	Количество требуемых десятичных знаков, чтобы всегда можно было различить разные значения (имеет смысл для всех типов чисел с плавающей точкой; только для стандарта C++11)	
<code>radix</code>	Целое число: база системы счисления представления (практически всегда 2). Плавающая точка: база экспоненциального представления	<code>FLT_RADIX</code>
<code>min_exponent</code>	Минимальная целая степень в системе счисления с основанием <code>radix</code>	<code>FLT_MIN_EXP</code> , ...

Окончание табл. 5.8

Член	Предназначение	Константы языка C
max_exponent	Максимальная положительная целая степень в системе счисления с основанием radix	FLT_MAX_EXP, ...
min_exponent10	Минимальная отрицательная целая степень в системе счисления с основанием 10	FLT_MIN_10_EXP, ...
max_exponent10	Максимальная положительная целая степень в системе счисления с основанием 10	FLT_MAX_10_EXP, ...
epsilon()	Разница между 1 и наименьшим значением, которое больше 1	FLT_EPSILON, ...
round_style	Способ округления	
round_error()	Величина максимальной ошибки округления (в соответствие со стандартом ISO/IEC 10967-1)	
has_infinity	Тип имеет представление для положительной бесконечности	
infinity()	Представление положительной бесконечности, если такое предусмотрено	
has_quiet_NaN	Тип имеет представление для не сигнализирующего “не числа”	
quiet_NaN()	Представление для не сигнализирующего “не числа”, если такое предусмотрено	
has_signaling_NaN	Тип имеет представление для сигнализирующего “не числа”	
signaling_NaN()	Представление для сигнализирующего “не числа”, если такое предусмотрено	
has_denorm	Допускает ли тип денормализованные значения (переменное количество битов в показателе степени)	
has_denorm_loss	Потеря точности распознается как потеря денормализации, а не как неточный результат	
denorm_min()	Минимальное положительное денормализованное значение	
traps	Реализована обработка прерывания	
tinyness_before	Перед округлением определяется малость числа	

Ниже приведена полная специализация числовых пределов для типа float, зависящая от платформы и демонстрирующая точные сигнатуры функций-членов.

```
namespace std {
    template<> class numeric_limits<float> {
    public:
        // да, специализация для числовых пределов
        // типа float существует
        static constexpr bool is_specialized = true;
```



```

inline constexpr float min() noexcept {
    return 1.17549435E-38F;
}

inline constexpr float max() noexcept {
    return 3.40282347E+38F;
}

inline constexpr float lowest() noexcept {
    return -3.40282347E+38F;
}

static constexpr int digits = 24;
static constexpr int digits10 = 6;
static constexpr int max_digits10 = 9;

static constexpr bool is_signed = true;
static constexpr bool is_integer = false;
static constexpr bool is_exact = false;
static constexpr bool is_bounded = true;
static constexpr bool is_modulo = false;
static constexpr bool is_iec559 = true;
static constexpr int radix = 2;

inline constexpr float epsilon() noexcept {
    return 1.19209290E-07F;
}

static constexpr float_round_style round_style
    = round_to_nearest;
inline constexpr float round_error() noexcept {
    return 0.5F;
}

static constexpr int min_exponent = -125;
static constexpr int max_exponent = +128;
static constexpr int min_exponent10 = -37;
static constexpr int max_exponent10 = +38;

static constexpr bool has_infinity = true;
inline constexpr float infinity() noexcept { return ...; }
static constexpr bool has_quiet_NaN = true;
inline constexpr float quiet_NaN() noexcept { return ...; }
static constexpr bool has_signaling_NaN = true;
inline constexpr float signaling_NaN() noexcept { return ...; }
static constexpr float_denorm_style has_denorm = denorm_absent;
static constexpr bool has_denorm_loss = false;
inline constexpr float denorm_min() noexcept { return min(); }

static constexpr bool traps = true;
static constexpr bool tinyness_before = true;
};
}

```

Отметим, что в стандарте C++11 все члены объявлены как `constexpr` (см. раздел 3.1.8). Например, функцию `max()` можно использовать там, где требуются выражения времени компиляции.

```
static const int ERROR_VALUE = std::numeric_limits<int>::max();
float a[std::numeric_limits<short>::max()];
```

До появления стандарта C++11 все члены класса, не являющиеся функциями, были константными и статическими, поэтому их значения определялись на этапе компиляции. Однако функции-члены были только `static`, так что вышеприведенные выражения были невозможны. Отметим также, что до появления стандарта C++11 функции `lowest()` и константы `max_digits10` не было, а вместо модификатора `constexpr` использовались пустые спецификации исключений (см. раздел 3.1.7).

Значения члена `round_style` приведены в табл. 5.9, значения члена `has_denorm` — в табл. 5.10. К сожалению, член `has_denorm` не называется `denorm_style`. Это произошло потому, что в процессе стандартизации было поздно изменять булев тип на значение перечисления. Однако член `has_denorm` можно использовать как булево значение, потому что стандарт гарантирует, что значение `denorm_absent` равно 0, что эквивалентно `false`, в то время как значение `denorm_present` равно 1, а `denorm_indeterminate` равно -1, что в обоих случаях эквивалентно значению `true`. Таким образом, член `has_denorm` можно рассматривать как булев индикатор того, что тип допускает денормализованные значения.

Таблица 5.9. Способ округления в классе `numeric_limits<>`

Способ округления	Смысл
<code>round_toward_zero</code>	Округление для нуля
<code>round_to_nearest</code>	Округление до ближайшего представимого числа
<code>round_toward_infinity</code>	Округление до положительной бесконечности
<code>round_toward_neg_infinity</code>	Округление до отрицательной бесконечности
<code>round_indeterminate</code>	Неопределимый

Таблица 5.10. Способ денормализации в классе `numeric_limits<>`

Способ денормализации	Смысл
<code>denorm_absent</code>	Тип не допускает денормализованные значения
<code>denorm_present</code>	Тип допускает денормализацию значения до ближайшего представимого числа
<code>denorm_indeterminate</code>	Неопределимый

Пример использования класса `numeric_limits<>`

Следующий пример демонстрирует возможное использование некоторых числовых пределов, например, максимальных значений для определенных типов и выяснение, есть ли знак у типа `char`.

```
// util/limits1.cpp

#include <iostream>
```

```

#include <limits>
#include <string>
using namespace std;

int main()
{
    // используется текстуальное представление булевских значений
    cout << boolalpha;

    // вывод максимального значения для целочисленных типов
    cout << "max(short): " << numeric_limits<short>::max() << endl;
    cout << "max(int): " << numeric_limits<int>::max() << endl;
    cout << "max(long): " << numeric_limits<long>::max() << endl;
    cout << endl;

    // вывод максимального значения для чисел с плавающей точкой
    cout << "max(float): "
        << numeric_limits<float>::max() << endl;
    cout << "max(double): "
        << numeric_limits<double>::max() << endl;
    cout << "max(long double): "
        << numeric_limits<long double>::max() << endl;
    cout << endl;

    // вывод, является ли тип char знаковым
    cout << "is_signed(char): "
        << numeric_limits<char>::is_signed << endl;
    cout << endl;

    // вывод, существуют ли
    // числовые пределы для типа string
    cout << "is_specialized(string): "
        << numeric_limits<string>::is_specialized << endl;
}

```

Вывод этой программы зависит от платформы. Вот один из возможных вариантов вывода:

```

max(short): 32767
max(int):   2147483647
max(long):  2147483647

max(float):      3.40282e+38
max(double):     1.79769e+308
max(long double): 1.79769e+308

is_signed(char): false

is_specialized(string): false

```

Последняя строка показывает, что для типа `string` числовые пределы не определены. Это логично, потому что строки не являются числовыми значениями. Однако этот пример показывает, что такую проверку можно выполнить для любого типа.

5.4. Свойства и утилиты типов

Почти все компоненты стандартной библиотеки языка C++ основаны на шаблонах. Для поддержки шаблонного программирования, которое иногда называют *метапрограммированием*, в библиотеке предусмотрены шаблонные утилиты, предназначенные как для программистов, так и для разработчиков библиотек.

Свойства типов (type traits), введенные в документе TR1 и расширенные в стандарте C++11, являются основой механизма для определения поведения, зависящего от типов. Их можно использовать для оптимизации кода, описывающего типы с особыми возможностями.

Могут оказаться полезными и другие утилиты, такие как оболочки для ссылок и функций.

5.4.1. Предназначение свойств типов

Свойство типа обеспечивает способ для работы с его атрибутами. Это шаблон, который на этапе компиляции создает конкретный тип или значение на основе одного или нескольких шаблонных аргументов, которые обычно являются типами.

Рассмотрим следующий пример:

```
template <typename T>
void foo (const T& val)
{
    if (std::is_pointer<T>::value) {
        std::cout << "foo() вызвана для указателя" << std::endl;
    }
    else {
        std::cout << "foo() вызвана для значения" << std::endl;
    }
    ...
}
```

Здесь свойство `std::is_pointer`, определенное в `<type_traits>`, используется для проверки того, является ли тип `T` типом указателя. Фактически шаблонный класс `is_pointer<>` дает либо тип `true_type`, либо тип `false_type`, для которых значение `::value` равно либо `true`, либо `false`. В результате, если полученный функцией `foo()` параметр `val` является указателем, она выведет строку

```
foo() вызвана для указателя
```

Отметим, однако, что следующий код является неправильным:

```
template <typename T>
void foo (const T& val)
{
    std::cout << (std::is_pointer<T>::value ? *val : val)
               << std::endl;
}
```

Причина заключается в том, что этот код генерируется как для `*val`, так и для `val`. Даже при передаче типа `int`, для которого выражение `is_pointer<T>::value` на этапе компиляции выдает значение `false`, код разворачивается в следующий оператор:

```
cout << (false ? *val : val) << endl;
```

Эта конструкция не компилируется, потому что выражение `*val` для типа `int` является некорректным.

Однако можно написать следующий код:

```
// реализация функции foo() для типов указателей:
template <typename T>
void foo_impl (const T& val, std::true_type)
{
    std::cout << "foo() вызвана для указателя на " << *val
               << std::endl;
}

// реализация функции foo() для типов не-указателей:
template <typename T>
void foo_impl (const T& val, std::false_type)
{
    std::cout << "foo() вызвана для значения " << val
               << std::endl;
}

template <typename T>
void foo (const T& val)
{
    foo_impl (val, std::is_pointer<T>());
}
```

Здесь в функции `foo()` выражение

```
std::is_pointer<T>()
```

во время компиляции дает тип `std::true_type` или `std::false_type`, который и определяет, какой из перегруженных вариантов функции `foo_impl()` будет конкретизирован.

Почему это лучше, чем два перегруженных варианта для функции `foo()`: один для обычных типов, а другой — для типов указателей?

```
template <typename T>
void foo (const T& val);    // общая реализация

template <typename T>
void foo<T*> (const T& val); // частичная специализация для указателей
```

Один из возможных ответов заключается в том, что иногда приходится создавать слишком много вариантов перегрузки. В целом мощь свойств типов обусловлена тем фактом, что они являются строительными конструкциями обобщенного кода. Этот факт можно продемонстрировать двумя примерами.

Гибкая перегрузка целочисленных типов

В работе [Becker:LibExt] Пете Беккер (Pete Becker) привел прекрасный пример, который мы слегка модифицировали. Допустим, у нас есть функция `foo()`, которую для аргументов, являющихся целыми числами и числами с плавающей точкой, необходимо реализовать по-разному. Обычно в таких случаях функцию перегружают для всех возможных типов целых чисел и чисел с плавающей точкой²².

```
void foo (short);           // целочисленная версия
void foo (unsigned short);
void foo (int);
...
void foo (float);           // версия для плавающей точки
void foo (double);
void foo (long double);
```

Это повторение не только утомительно, но и создает проблему, которая заключается в том, что для новых целочисленных типов и типов чисел с плавающей точкой данный код может не работать, независимо от того, предусмотрены ли они стандартом, как `long long`, или пользователем.

С помощью свойств типа можно написать следующий код:

```
template <typename T>
void foo_impl (T val, true_type);    // версия для целочисленных типов

template <typename T>
void foo_impl (T val, false_type);   // версия для плавающей точки

template <typename T>
void foo (T val)
{
    foo_impl (val, std::is_integral<T>());
}
```

Таким образом, мы имеем две реализации — одну для целочисленных типов, а другую для типов чисел с плавающей точкой — и выбираем правильную реализацию в зависимости от того, какой тип создает шаблонный класс `std::is_integral<>`.

Обработка общего типа

Другим примером, демонстрирующим полезность свойств типов, является потребность в обработке общего типа для двух или более типов. В этом случае значения двух разных типов можно рассматривать как значения общего типа. Например, таким типом может быть тип минимального из двух значений или тип суммы двух значений разных типов. Если не предусмотреть такой тип, то при реализации функции, определяющей минимальное из двух значений разных типов, становится непонятным, каким должен быть тип возвращаемого значения.

²² В соответствии со стандартом языка C++ термин *целочисленный тип* (integral type) относится также к типам `bool` и символьным типам, но в этом примере это не имеет значения.

```
template <typename T1, typename T2>
??? min (const T1& x, const T2& y);
```

Используя свойства типов, при объявлении общего типа можно просто использовать класс `std::common_type<>`.

```
template <typename T1, typename T2>
typename std::common_type<T1,T2>::type min (const T1& x, const T2& y);
```

Например, выражение `std::common_type<T1,T2>::type` выдает `int`, если оба аргумента имеют типы `int`; `long`, если один из них имеет тип `int`, а другой `long`; или `std::string`, если один из них является строкой, а второй — строковым литералом (тип `const char*`).

Как это работает? Здесь просто используется правило, реализованное для оператора `?:`, который возвращает тип результата в зависимости от типа обоих операндов. Фактически класс `std::common_type<>` реализован следующим образом:

```
template <typename T1, typename T2>
struct common_type<T1,T2> {
    typedef decltype(true ? declval<T1>() : declval<T2>()) type;
};
```

где `decltype` — новое ключевое слово в стандарте C++11 (см. раздел 3.1.11), предназначенное для получения типа выражения, а `declval<>` — вспомогательное свойство для предоставления объявленного значения переданного типа без его вычисления (генерации для него `rvalue`-ссылки).

Таким образом, если оператор `?:` может определить общий тип, класс `common_type<>` выдает его. Если нет, можно предусмотреть перегрузку `common_type<>` (именно этот прием использован в заголовке `<chrono>` для комбинирования продолжительности времени; см. раздел 5.7.2).

5.4.2. Подробное описание свойств типов

Свойства типов обычно определяются в заголовочном файле `<type_traits>`.

(Унарные) предикаты типов

Как указывалось в разделе 5.4.1, предикаты типов дают тип `std::true_type`, если заданное свойство имеет место, и `std::false_type`, если нет. Эти типы являются специализациями вспомогательного класса `std::integral_constant`, поэтому их соответствующие значения члена `value` равны `true` или `false`.

```
namespace std {
    template <typename T, T val>
    struct integral_constant {
        static constexpr T value = val;
        typedef T value_type;
        typedef integral_constant<T,val> type;
        constexpr operator value_type() {
            return value;
        }
    }
```

```
};
typedef integral_constant<bool,true> true_type;
typedef integral_constant<bool,false> false_type;
}
```

Предикаты для всех типов приведены в табл. 5.11, а свойства, уточняющие детали классов, — в табл. 5.12.

Таблица 5.11. Свойства для проверки характеристик типов

Свойство	Действие
<code>is_void<T></code>	Тип <code>void</code>
<code>is_integral<T></code>	Целочисленный тип (включая <code>bool</code> , <code>char</code> , <code>char16_t</code> , <code>char32_t</code> , <code>wchar_t</code>)
<code>is_floating_point<T></code>	Тип чисел с плавающей точкой (<code>float</code> , <code>double</code> , <code>long double</code>)
<code>is_arithmetic<T></code>	Целочисленный тип (включая <code>bool</code> и символы) или тип числа с плавающей точкой
<code>is_signed<T></code>	Арифметический тип со знаком
<code>is_unsigned<T></code>	Арифметический тип без знака
<code>is_const<T></code>	Квалифицированный модификатором <code>const</code>
<code>is_volatile<T></code>	Квалифицированный модификатором <code>volatile</code>
<code>is_array<T></code>	Обычный массив (не тип <code>std::array</code>)
<code>is_enum<T></code>	Перечисление
<code>is_union<T></code>	Объединение
<code>is_class<T></code>	Класс/структура, но не объединение
<code>is_function<T></code>	Функция
<code>is_reference<T></code>	Lvalue- или rvalue-ссылка
<code>is_lvalue_reference<T></code>	Lvalue-ссылка
<code>is_rvalue_reference<T></code>	Rvalue-ссылка
<code>is_pointer<T></code>	Указатель (включая указатель на функцию, но не указатель на нестатическую функцию-член)
<code>is_member_pointer<T></code>	Указатель на нестатическую функцию-член
<code>is_member_object_pointer<T></code>	Указатель на нестатический член-данные
<code>is_member_function_pointer<T></code>	Указатель на нестатическую функцию-член
<code>is_fundamental<T></code>	<code>void</code> , целочисленный (включая <code>bool</code> и символы), числа с плавающей точкой или <code>std::nullptr_t</code>
<code>is_scalar<T></code>	Целочисленный (включая <code>bool</code> и символы), числа с плавающей точкой, перечисление, указатель, указатель на член, <code>std::nullptr_t</code>

Окончание табл. 5.11

Свойство	Действие
<code>is_object<T></code>	Любой тип, за исключением <code>void</code> , функции или ссылки
<code>is_compound<T></code>	Массив, перечисление, объединение, класс, функция, ссылка или указатель
<code>is_trivial<T></code>	Скаляр, тривиальный класс или массивы объектов таких типов
<code>is_trivially_copyable<T></code>	Скаляр, тривиально копируемый класс или массивы объектов таких типов
<code>is_standard_layout<T></code>	Скаляр, стандартный макетный класс (standard layout class) или массивы объектов этих типов
<code>is_pod<T></code>	Простой тип данных (тип, для которого копирование объектов выполняет функция <code>memcpy()</code>)
<code>is_literal_type<T></code>	Скаляр, ссылка, класс или массивы объектов таких типов

Таблица 5.12. Свойства для проверки атрибутов классов

Свойство	Действие
<code>is_empty<T></code>	Класс без членов, виртуальные функции-члены или виртуальные базовые классы
<code>is_polymorphic<T></code>	Класс с (производной) виртуальной функцией-членом
<code>is_abstract<T></code>	Абстрактный класс (класс, содержащий по крайней мере одну чисто виртуальную функцию)
<code>has_virtual_destructor<T></code>	Класс с виртуальным деструктором
<code>is_default_constructible<T></code>	Класс, допускающий конструктор по умолчанию
<code>is_copy_constructible<T></code>	Класс, допускающий копирующий конструктор
<code>is_move_constructible<T></code>	Класс, допускающий перемещающий конструктор
<code>is_copy_assignable<T></code>	Класс, допускающий копирующее присваивание
<code>is_move_assignable<T></code>	Класс, допускающий перемещающее присваивание
<code>is_destructible<T></code>	Класс с вызываемым деструктором (не удаленным, не защищенным и не закрытым)
<code>is_trivially_default_constructible<T></code>	Класс, допускающий тривиальный конструктор по умолчанию
<code>is_trivially_copy_constructible<T></code>	Класс, допускающий тривиальный копирующий конструктор по умолчанию
<code>is_trivially_move_constructible<T></code>	Класс, допускающий тривиальный перемещающий конструктор
<code>is_trivially_copy_assignable<T></code>	Класс, допускающий тривиальное копирующее присваивание

Свойство	Действие
<code>is_trivially_move_assignable<T></code>	Класс с тривиальным перемещающим присваиванием
<code>is_trivially_destructible<T></code>	Класс с тривиальным вызываемым деструктором
<code>is_nothrow_default_constructible<T></code>	Класс, допускающий конструктор по умолчанию, который не генерирует исключения
<code>is_nothrow_copy_constructible<T></code>	Класс, допускающий копирующий конструктор по умолчанию, который не генерирует исключения
<code>is_nothrow_move_constructible<T></code>	Класс, допускающий перемещающий конструктор, который не генерирует исключения
<code>is_nothrow_copy_assignable<T></code>	Класс, допускающий копирующее присваивание, не генерирующее исключение
<code>is_nothrow_move_assignable<T></code>	Класс, допускающий перемещающее присваивание, не генерирующее исключение
<code>is_nothrow_destructible<T></code>	Класс, допускающий вызываемый деструктор, не генерирующий исключение

Тип `bool` и все символьные типы (`char`, `char16_t`, `char32_t` и `wchar_t`) относятся к целочисленным типам, а тип `std::nullptr_t` (см. раздел 3.1.1) считается элементарным.

В большинстве (хотя и не все) свойства являются унарными. Иначе говоря, они используют только один шаблонный аргумент. Например, предикат `is_const<>` проверяет, является ли передаваемый тип константным.

```
is_const<int>::value           // false
is_const<const volatile int>::value // true
is_const<int* const>::value    // true
is_const<const int*>::value     // false
is_const<const int&>::value     // false
is_const<int[3]>::value         // false
is_const<const int[3]>::value    // true
is_const<int[]>::value         // false
is_const<const int[]>::value    // true
```

Отметим, что неконстантный указатель или ссылка на константный тип являются неконстантными, в то время как обычный массив константных элементов считается константным²³.

Свойства, проверяющие семантику копирования и перемещения, проверяют только то, возможны ли соответствующие выражения. Например, тип с копирующим конструктором, имеющим константный аргумент, но не имеющий перемещающего конструктора, все же имеет семантику перемещения.

Свойство типа `is_nothrow...` особенно удобно для формулировки спецификаций noexcept (см. раздел 3.1.7).

²³ Корректность этого утверждения в настоящее время является предметом дискуссий в группе по стандартизации ядра языка.

Свойства отношений между типами

В табл. 5.13 перечислены свойства типа, допускающие проверку отношений между типами. В частности, можно проверить, предусмотрены ли конструкторы и операторы присваивания в классе.

Таблица 5.13. Свойства для проверки отношений между типами

Свойство	Действие
<code>is_same<T1, T2></code>	Типы <i>T1</i> и <i>T2</i> являются одинаковыми (с учетом квалификаторов <code>const/volatile</code>)
<code>is_base_of<T, D></code>	Тип <i>T</i> является базовым классом по отношению к типу <i>D</i>
<code>is_convertible<T, T2></code>	Тип <i>T</i> может быть преобразован в тип <i>T2</i>
<code>is_constructible<T, Args...></code>	Можно инициализировать тип <i>T</i> типами <i>Args</i>
<code>is_trivially_constructible<T, Args...></code>	Можно тривиально инициализировать тип <i>T</i> типами <i>Args</i>
<code>is_nothrow_constructible<T, Args...></code>	Инициализация типа <i>T</i> типами <i>Args</i> не генерирует исключений
<code>is_assignable<T, T2></code>	Можно присваивать тип <i>T2</i> типу <i>T</i>
<code>is_trivially_assignable<T, T2></code>	Можно тривиально присваивать тип <i>T2</i> типу <i>T</i>
<code>is_nothrow_assignable<T, T2></code>	Присваивание типа <i>T2</i> типу <i>T</i> не генерирует исключений
<code>uses_allocator<T, Alloc></code>	Тип <i>Alloc</i> может быть преобразован в тип <i>T::allocator_type</i>

Отметим, что тип `int` допускает как `lvalue`-, так и `rvalue`-значения. Поскольку присваивание

```
42 = 77;
```

невозможно, предикат `is_assignable<>` для типов, имеющих первый шаблонный аргумент, не являющийся классом, всегда возвращает значение `false_type`. Однако если первым шаблонным аргументом является класс, то этот предикат возвращает истинное значение, потому что существует старое правило, позволяющее вызывать функции-члены из `rvalue`-значений классов²⁴. Например:

```
is_assignable<int,int>::value           // false
is_assignable<int&,int>::value          // true
is_assignable<int&&,int>::value          // false
is_assignable<long&,int>::value         // true
is_assignable<int&,void*>::value         // false
is_assignable<void*,int>::value          // false
is_assignable<const char*,std::string>::value // false
is_assignable<std::string,const char*>::value // true
```

²⁴ Благодарю за это замечание Даниэля Крюглера.

Рассмотрим примеры применения свойства `is_constructible<>`:

```
is_constructible<int>::value           // true
is_constructible<int,int>::value       // true
is_constructible<long,int>::value      // true
is_constructible<int,void*>::value     // false
is_constructible<void*,int>::value     // false
is_constructible<const char*,std::string>::value // false
is_constructible<std::string,const char*>::value // true
is_constructible<std::string,const char*,int,int>::value // true
```

Свойство `std::uses_allocator<>` определено в заголовке `<memory>` (см. раздел 19.1).

Модификаторы типа

Свойства, перечисленные в табл. 5.14, позволяют модифицировать типы.

Все модифицирующие свойства добавляют атрибут типа, которого в нем не было, или удаляют ранее существовавший атрибут. Например, тип `int` можно лишь расширить:

```
typedef int T;
add_const<T>::type           // const int
add_lvalue_reference<T>::type // int&
add_rvalue_reference<T>::type // int&&
add_pointer<T>::type         // int*
make_signed<T>::type         // int
make_unsigned<T>::type       // unsigned int
remove_const<T>::type        // int
remove_reference<T>::type    // int
remove_pointer<T>::type      // int
```

а тип `const int&` можно как сузить, так и расширить:

```
typedef const int& T;
add_const<T>::type           // const int&
add_lvalue_reference<T>::type // const int&
add_rvalue_reference<T>::type // const int& (да, lvalue остается lvalue)
add_pointer<T>::type         // const int*
make_signed<T>::type         // непредсказуемое поведение
make_unsigned<T>::type       // непредсказуемое поведение
remove_const<T>::type        // const int&
remove_reference<T>::type    // const int
remove_pointer<T>::type      // const int&
```

Напомним, что ссылка на константный тип не является константным типом, поэтому у него нельзя отнять модификатор `const`. Отметим, что предикат `add_pointer<>` подразумевает применение предиката `remove_reference<>`. Однако предикаты `make_signed<>` и `make_unsigned<>` требуют, чтобы аргументы были либо целочисленными, либо перечислениями, исключая `bool`, поэтому передача ссылок приводит к непредсказуемым последствиям.

Таблица 5.14. Свойства для модификации типов

Свойство	Действие
<code>remove_const<T></code>	Соответствующий тип без <code>const</code>
<code>remove_volatile<T></code>	Соответствующий тип без <code>volatile</code>
<code>remove_cv<T></code>	Соответствующий тип без <code>const</code> и <code>volatile</code>
<code>add_const<T></code>	Соответствующий тип <code>const</code>
<code>add_volatile<T></code>	Соответствующий тип <code>volatile</code>
<code>add_cv<T></code>	Соответствующий тип <code>const volatile</code>
<code>make_signed<T></code>	Соответствующий тип со знаком, не являющийся ссылочным
<code>make_unsigned<T></code>	Соответствующий тип без знака, не являющийся ссылочным
<code>remove_reference<T></code>	Соответствующий тип, не являющийся ссылочным
<code>add_lvalue_reference<T></code>	Соответствующий ссылочный <code>lvalue</code> -тип (<code>rvalue</code> -тип становится <code>lvalue</code> -типом)
<code>add_rvalue_reference<T></code>	Соответствующий ссылочный <code>rvalue</code> -тип (<code>lvalue</code> -тип остается <code>lvalue</code> -типом)
<code>remove_pointer<T></code>	Тип объекта, на который ссылается указатель (если предикат применяется не к указателю, тип не изменяется)
<code>add_pointer<T></code>	Тип указателя для соответствующего типа, не являющегося ссылочным

Отметим, что свойство `add_lvalue_reference<>` преобразует `rvalue`-ссылку в `lvalue`-ссылку, а `add_rvalue_reference<>` *не* преобразует `lvalue`-ссылку в `rvalue`-ссылку (тип остается неизменным). Следовательно, чтобы конвертировать `lvalue`-ссылку в `rvalue`-ссылку, необходимо вызвать

```
add_rvalue_reference<remove_reference<T>::type>::type
```

Другие свойства типов

Остальные свойства типов приведены в табл. 5.15. К ним относятся запросы для специальных свойств, проверки отношений между типами и более сложные преобразования типов.

Таблица 5.15. Другие свойства типов

Свойства	Действие
<code>rank<T></code>	Количество измерений массива типа <code>T</code> (или 0)
<code>extent<T, I=0></code>	Расширение размерности <code>I</code> (или 0)
<code>remove_extent<T></code>	Тип элементов массива (в противном случае — тот же самый тип)
<code>remove_all_extents<T></code>	Тип элементов для многомерных массивов (в противном случае — тот же самый тип)
<code>underlying_type<T></code>	Тип, лежащий в основе перечисления (см. раздел 3.1.13)

Свойства	Действие
<code>decay<T></code>	Преобразование в соответствующий тип “по значению”
<code>enable_if<B, T=void></code>	Возвращает тип <i>T</i> , только если значение <i>B</i> типа <code>bool</code> равно <code>true</code>
<code>conditional<B, T, F></code>	Возвращает тип <i>T</i> , если значение <i>B</i> типа <code>bool</code> равно <code>true</code> , в противном случае возвращает тип <i>F</i>
<code>common_type<T1, ...></code>	Общий тип для всех передаваемых типов
<code>result_of<F, ArgTypes></code>	Тип результата вызова <i>F</i> с аргументами <i>ArgTypes</i>
<code>alignment_of<T></code>	Эквивалент <code>alignof(T)</code>
<code>aligned_storage<Len></code>	Тип <i>Len</i> байтов с выравниванием по умолчанию
<code>aligned_storage<Len, Align></code>	Тип <i>Len</i> байтов, выровненных в соответствии с делителем <i>Align</i> типа <code>size_t</code>
<code>aligned_union<Len, Types...></code>	Тип <i>Len</i> байтов, выровненный для объединения <i>Types...</i>

Свойства, работающие с предикатами `rank` и `extent`, позволяют обрабатывать (многомерные) массивы. Например:

```
rank<int>::value           // 0
rank<int[]>::value         // 1
rank<int[5]>::value        // 1
rank<int[][7]>::value       // 2
rank<int[5][7]>::value      // 2
extent<int>::value         // 0
extent<int[]>::value       // 0
extent<int[5]>::value      // 5
extent<int[][7]>::value     // 0
extent<int[5][7]>::value    // 5
extent<int[][7], 1>::value  // 7
extent<int[5][7], 1>::value // 7
extent<int[5][7], 2>::value // 0
remove_extent<int>::type   // int
remove_extent<int[]>::type // int
remove_extent<int[5]>::type // int
remove_extent<int[][7]>::type // int[7]
remove_extent<int[5][7]>::type // int[7]
remove_all_extents<int>::type // int
remove_all_extents<int[]>::type // int
remove_all_extents<int[5]>::type // int
remove_all_extents<int[][7]>::type // int
remove_all_extents<int[5][7]>::type // int
```

Свойство `decay<>` предоставляет возможность конвертировать тип *T* в соответствующий ему тип, когда этот тип передается по значению. Таким образом, оно конвертирует типы массивов и функций в указатели, а также `lvalue`-значения в `rvalue`-значения, включая удаление модификаторов `const` и `volatile`. Примеры их использования см. в разделе 5.1.1.

Как указано в разделе 5.4.1, свойство `common_type<>` возвращает общий тип для всех передаваемых типов (можно передавать один, два аргумента и больше).

5.4.3. Обертки для ссылок

Класс `std::reference_wrapper<>`, объявленный в заголовке `<functional>`, используется в основном для передачи ссылок в шаблонные функции, которые принимают параметры по значению. Для заданного типа `T` этот класс предоставляет функцию `ref()` для неявного преобразования в `T&` и `cref()` для неявного преобразования в `const T&`. Это обычно позволяет шаблонным функциям работать со ссылками без специализации.

Например, после объявления

```
template <typename T>
void foo (T val);
```

в вызове

```
int x;
foo (std::ref(x));
```

тип `T` становится `int&`, а в вызове

```
int x;
foo (std::cref(x));
```

тип `T` становится `const int&`.

Эта функциональная возможность используется в разных местах стандартной библиотеки языка C++. Соответствующие примеры приведены ниже.

- Функция `make_pair()` использует ее для того, чтобы создавать объекты класса `pair<>`, состоящие из ссылок (см. раздел 5.1.1).
- Функция `make_tuple()` использует ее для того, чтобы создавать объекты класса `tuple<>`, состоящие из ссылок (см. раздел 5.1.2).
- Связыватели используют ее для связывания ссылок (см. раздел 10.2.2).
- Потоки используют ее для передачи аргументов по ссылке (см. раздел 18.2.2).

Отметим также, что класс `reference_wrapper` позволяет использовать ссылки в качестве объектов первого класса, таких как тип элементов массива или контейнера STL.

```
std::vector<MyClass&> coll; // Ошибка
std::vector<std::reference_wrapper<MyClass>> coll; // OK
```

Детали описаны в разделе 7.11.

5.4.4. Обертки функциональных типов

Класс `std::function<>`, объявленный в заголовке `<functional>`, предоставляет полиморфные обертки, обобщающие понятие указателя на функцию. Этот класс позволяет использовать *вызываемые объекты* (функции, функции-члены, функторы и лямбда-функции; см. раздел 4.4) в качестве объектов первого класса. Например:

```
void func (int x, int y);

// инициализация коллекции заданий:
```

```
std::vector<std::function<void(int,int)>> tasks;
tasks.push_back(func);
tasks.push_back([] (int x, int y) {
    ...
});
// вызов каждой задачи:
for (std::function<void(int,int)> f : tasks) {
    f(33,66);
}
```

Для вызова каждой задачи можно также просто выполнить следующий код.

```
// вызов каждой задачи:
for (auto f : tasks) {
    f(33,66);
}
```

При использовании функций-членов объект, из которого они вызываются, должен передаваться как первый аргумент.

```
class C {
public:
    void memfunc (int x, int y) const;
};

std::function<void(const C&,int,int)> mf;
mf = &C::memfunc;
mf(C(),42,77);
```

Другое применение этой функциональной возможности — объявление функций, возвращающих лямбда-функции (см. раздел 3.1.10).

Отметим, что вызов функции без указания *цели* (т.е. конкретного вызываемого объекта) приводит к генерированию исключения `std::bad_function_call` (см. раздел 4.3.1).

```
std::function<void(int,int)> f;
f(33,66); // генерирование исключения std::bad_function_call
```

5.5. Вспомогательные функции

Стандартная библиотека языка C++ содержит несколько вспомогательных функций, например, вычисляющих минимум и максимум, обменивающих значения и предоставляющих дополнительные операторы сравнения.

5.5.1. Вычисление минимума и максимума

В табл. 5.16 приведены вспомогательные функции, определенные в заголовке `<algorithm>` и вычисляющие максимум и/или минимум из двух или более функций. Все функции `minmax()` и все функции для списков инициализации предусмотрены лишь стандартом C++11.

Таблица 5.16. Операции для вычисления минимума и максимума

Операция	Действие
<code>min(a, b)</code>	Возвращает минимум из <i>a</i> и <i>b</i> , сравнивая их с помощью оператора <
<code>min(a, b, cmp)</code>	Возвращает минимум из значений <i>a</i> и <i>b</i> , сравнивая их с помощью функции <i>cmp</i>
<code>min(initlist)</code>	Возвращает минимум из списка <i>initlist</i> , выполняя сравнение с помощью оператора <
<code>min(initlist, cmp)</code>	Возвращает минимум из списка <i>initlist</i> , выполняя сравнение с помощью функции <i>cmp</i>
<code>max(a, b)</code>	Возвращает максимум из значений <i>a</i> и <i>b</i> , выполняя сравнение с помощью оператора <
<code>max(a, b, cmp)</code>	Возвращает максимум из списка <i>initlist</i> , выполняя сравнение с помощью функции <i>cmp</i>
<code>max(initlist)</code>	Возвращает максимум из списка <i>initlist</i> , выполняя сравнение с помощью оператора <
<code>max(initlist, cmp)</code>	Возвращает максимум из списка <i>initlist</i> , выполняя сравнение с помощью функции <i>cmp</i>
<code>minmax(a, b)</code>	Возвращает максимум и минимум из значений <i>a</i> и <i>b</i> , сравнивая их с помощью оператора <
<code>minmax(a, b, cmp)</code>	Возвращает максимум и минимум из значений <i>a</i> и <i>b</i> , сравнивая их с помощью функции <i>cmp</i>
<code>minmax(initlist)</code>	Возвращает максимум и минимум из списка <i>initlist</i> , сравнивая их с помощью оператора <
<code>minmax(initlist, cmp)</code>	Возвращает максимум и минимум из списка <i>initlist</i> , сравнивая их с помощью функции <i>cmp</i>

Функция `minmax()` возвращает объект класса `pair<>` (см. раздел 5.1.1), в котором первым значением является минимум, а вторым — максимум. Для версий с двумя аргументами функции `min()` и `max()` возвращают первый элемент, если оба значения равны. Для списков инициализации функции `min()` и `max()` возвращают первый из нескольких минимальных или максимальных элементов. Функция `minmax()` возвращает пару, состоящую из чисел *a* и *b* для двух равных аргументов, и первый минимум, но последний максимум для списка инициализации. Однако хороший стиль программирования требует не полагаться на эти особенности функций.

Версии, получающие два значения, возвращают ссылку; версии, получающие список инициализации, возвращают копии значений.

```
namespace std {
    template <typename T>
        const T& min (const T& a, const T& b);
    template <typename T>
        T min (initializer_list<T> initlist);
    ...
}
```

Причина заключается в том, что в данном случае используется внутренняя временная переменная, поэтому возвращение ссылки привело бы к появлению висячего указателя.

Обе функции в качестве дополнительного аргумента принимают критерий сравнения.

```
namespace std {
    template <typename T, typename Compare>
        const T& min (const T& a, const T& b, Compare cmp);
    template <typename T, typename Compare>
        T min (initializer_list<T> initlist, Compare cmp);
    ...
}
```

Аргумент сравнения может быть функцией или функциональным объектом (см. раздел 6.10), сравнивающим два аргумента и возвращающим результат проверки того, является ли первый аргумент меньшим второго в определенном порядке.

Следующий пример демонстрирует использование функции для вычисления максимума, получающей в качестве аргумента специальную функцию для сравнения:

```
// util/minmax1.cpp

#include <algorithm>

// функция, сравнивающая два указателя путем сравнения
// значений, на которые они ссылаются
bool int_ptr_less (int* a, int* b)
{
    return *a < *b;
}

int main()
{
    int x = 17;
    int y = 42;
    int z = 33;
    int* px = &x;
    int* py = &y;
    int* pz = &z;

    // вызываем функцию max() со специальной функцией сравнения
    int* pmax = std::max (px, py, int_ptr_less);

    // вызываем функцию minmax() для списка инициализации
    // со специальной функцией сравнения
    std::pair<int*,int*> extremes = std::minmax ({px, py, pz},
                                                int_ptr_less);
    ...
}
```

В качестве альтернативы для задания критерия сравнения можно использовать новое языковое средство — лямбда-функцию, а с помощью ключевого слова `auto` избежать явного объявления типа возвращаемого значения.

```
auto extremes = std::minmax ({px, py, pz}, [](int*a, int*b) {
                                return *a < *b;
                            });
```

Определения функций `min()` и `max()` требуют, чтобы оба типа совпадали. Таким образом, их нельзя вызывать для объектов, имеющих разные типы.

```
int i;
long l;
...
std::max(i,l);    // ОШИБКА: типы аргументов не совпадают
std::max({i,l}); // ОШИБКА: типы аргументов не совпадают
```

Однако можно явно уточнить тип с помощью шаблонных аргументов (задавая тем самым тип возвращаемого значения).

```
std::max<long>(i,l);    // OK
std::max<long>({i,l}); // OK
```

5.5.2. Обмен двух значений

Функция `swap()` обменивает два объекта. Общая реализации функции `swap()` определена в заголовке `<utility>` следующим образом²⁵:

```
namespace std {
    template <typename T>
    inline void swap(T& a, T& b) ... {
        T tmp(std::move(a));
        a = std::move(b);
        b = std::move(tmp);
    }
}
```

Таким образом, если есть возможность, значения перемещаются или присваиваются путем перемещения (см. детали семантики перемещения в разделе 3.1.5). До появления стандарта C++11 значения присваивались или копировались всегда.

Используя эту функцию, можно обменять содержимое двух произвольных переменных `x` и `y`, выполнив вызов

```
std::swap(x,y);
```

Разумеется, этот вызов возможен только в том случае, если тип параметра допускает семантику перемещения или копирования.

Отметим, что функция `swap()` предоставляет спецификацию исключения (вот почему в предыдущих объявлениях использовался эллипсис `...`). Спецификация исключения функции `swap()` в общем случае имеет следующий вид²⁶:

²⁵ До версии C++11 функция `swap()` была определена в заголовочном файле `<algorithm>`.

²⁶ Подробности, касающиеся ключевого слова `noexcept`, изложены в разделе 3.1.7, а свойства типов, использованные в данном примере, описаны в разделе 5.4.2.

```
noexcept(is_nothrow_move_constructible<T>::value &&
         is_nothrow_move_assignable<T>::value)
```

В соответствии со стандартом C++11 стандартная библиотека языка C++ предусматривает перегрузку для массивов.

```
namespace std {
    template <typename T, size_t N>
        void swap (T (&a)[N], T (&b)[N])
            noexcept(noexcept(swap(*a,*b)));
}
```

Большое преимущество функции `swap()` заключается в том, что она позволяет создавать специальные реализации для более сложных типов с помощью специализации шаблонов или перегрузки функций. Эти специальные реализации помогают сэкономить время, поменяв местами внутренние члены, а не присваивая объекты. Это касается, например, всех стандартных контейнеров (см. раздел 7.1.2) и строк (см. раздел 13.2.8). Например, реализация функции `swap()` для простого контейнера, содержащая только массив и количество его элементов, может выглядеть примерно так:

```
class MyContainer {
private:
    int* elems;    // динамический массив элементов
    int numElems;  // количество элементов
public:
    ...
    // реализация функции swap()
    void swap(MyContainer& x) {
        std::swap(elems,x.elems);
        std::swap(numElems,x.numElems);
    }
    ...
};

// перегруженная глобальная версия функции swap() для этого типа
inline void swap (MyContainer& c1, MyContainer& c2)
    noexcept(noexcept(c1.swap(c2)))
{
    c1.swap(c2); // calls implementation of swap()
}
```

Итак, вызывая функцию `swap()` вместо непосредственного обмена значений, можно значительно повысить быстродействие программы. Если возникает возможность повысить производительность, всегда следует предусматривать специализацию функции `swap()` для своих типов.

Подчеркнем, что оба типа должны совпадать:

```
int i;
long l;
std::swap(i,l);    // ОШИБКА: типы аргументов не совпадают
int a1[10];
int a3[11];
std::swap(a1,a3);  // ОШИБКА: массивы имеют разные типы (разные размеры)
```

5.5.3. Вспомогательные операторы сравнения

Четыре шаблонных функции определяют операторы сравнения `!=`, `>`, `<=` и `>=` с помощью операторов `==` и `<`. Эти функции объявлены в заголовке `<utility>` и обычно определяются следующим образом:

```
namespace std {
    namespace rel_ops {
        template <typename T>
        inline bool operator!= (const T& x, const T& y) {
            return !(x == y);
        }
        template <typename T>
        inline bool operator> (const T& x, const T& y) {
            return y < x;
        }
        template <typename T>
        inline bool operator<= (const T& x, const T& y) {
            return !(y < x);
        }
        template <typename T>
        inline bool operator>= (const T& x, const T& y) {
            return !(x < y);
        }
    }
}
```

Для использования этих функций необходимо лишь определить операторы `<` и `==`. Использование пространства имен `std::rel_ops` определяет другие операторы сравнения автоматически. Рассмотрим пример:

```
#include <utility>

class X {
public:
    bool operator== (const X& x) const;
    bool operator< (const X& x) const;
    ...
};

void foo()
{
    using namespace std::rel_ops; // открывает доступ к операторам !=, > и др.
    X x1, x2;
    ...
    if (x1 != x2) { // OK
        ...
    }
    if (x1 > x2) { // OK
        ...
    }
}
```

Эти операторы определены в подпространстве имен `rel_ops` пространства имен `std`. Они находятся в отдельном пространстве имен, поэтому операции сравнения, определенные пользователем в глобальном пространстве имен, не создадут конфликта, даже если все идентификаторы в пространстве имен `std` станут глобальными после использования директивы

```
using namespace std; // операции не находятся в глобальной области видимости
```

С другой стороны, пользователи, желающие получить явный доступ к этим операциям, могут реализовать их, не используя правила неявного поиска.

```
using namespace std::rel_ops; // операции находятся
                             // в глобальной области видимости
```

Некоторые реализации определяют операции с двумя разными типами аргументов.

```
namespace std {
    namespace rel_ops {
        template <typename T1, typename T2>
        inline bool operator!=(const T1& x, const T2& y) {
            return !(x == y);
        }
        ...
    }
}
```

Преимущество такой реализации заключается в том, что в ней типы операндов могут быть разными. Достаточно лишь, чтобы их можно было сравнивать. Следует, однако, отметить, что такая реализация в стандартной библиотеке языка C++ не определена. Следовательно, такой код не будет переносимым.

5.6. Арифметика рациональных чисел на этапе компиляции

Класс `ratio<>`

В соответствии со стандартом C++11 стандартная библиотека языка C++ предусматривает интерфейс для дробей и арифметических операций над ними, выполняемых на этапе компиляции. Пропитируем [N2661:Chrono] (с небольшими изменениями):²⁷

Средство для работы с дробями является универсальным механизмом, предложенным Уолтером Е. Брауном, для простого и безопасного вычисления рациональных чисел на этапе компиляции. Класс `ratio` перехватывает все ошибки (например, деление на ноль и переполнение) на этапе компиляции. Он используется в библиотеках, связанных с измерениями продолжительности и времени [см. раздел 5.7], для эффективного

²⁷ Благодарю Уолтера Е. Брауна (Walter E. Brown), Говарда Хиннанта (Howard Hinnant), Джеффа Гарланда (Jeff Garland), и Марка Патерно (Marc Paterno) за любезное разрешение процитировать работу [N2661:Chrono] здесь и в следующем разделе, посвященном функциям работы со временем.

создания единиц времени. Его можно также применять в других “количественных” библиотеках (как стандартных, так и пользовательских) или в любом другом месте, где имеется рациональная константа, известная на этапе компиляции. Использование этого класса позволяет существенно снизить вероятность переполнения на этапе выполнения программы, поскольку дроби и результаты выполнения арифметических операций над ними всегда сокращаются до несократимой дроби.

Средство для работы с дробями описано в заголовке `<ratio>` в виде класса `ratio<>`, определенного следующим образом:

```
namespace std {
    template <intmax_t N, intmax_t D = 1>
    class ratio {
    public:
        static constexpr intmax_t num;
        static constexpr intmax_t den;

        typedef ratio<num,den> type;
    };
}
```

Тип `intmax_t` означает целочисленный тип со знаком, позволяющий представить любое целое число со знаком. Он определен в заголовке `<cstdint>` или `<stdint.h>` и имеет не менее 64 битов. Числитель и знаменатель являются открытыми и автоматически сокращаются до несократимой дроби. Например:

```
// util/ratio1.cpp

#include <ratio>
#include <iostream>
using namespace std;

int main()
{
    typedef ratio<5,3> FiveThirds;
    cout << FiveThirds::num << "/" << FiveThirds::den << endl;

    typedef ratio<25,15> AlsoFiveThirds;
    cout << AlsoFiveThirds::num << "/" << AlsoFiveThirds::den << endl;

    ratio<42,42> one;
    cout << one.num << "/" << one.den << endl;

    ratio<0> zero;
    cout << zero.num << "/" << zero.den << endl;

    typedef ratio<7,-3> Neg;
    cout << Neg::num << "/" << Neg::den << endl;
}
```

Программа выводит следующие результаты:

5/3
5/3
1/1
0/1
-7/3

В табл. 5.17 перечислены операции над рациональными числами, которые можно выполнять на этапе компиляции. Четыре основных арифметических операции, +, -, * и /, определены как `ratio_add`, `ratio_subtract`, `ratio_multiply` и `ratio_divide`. Результат имеет тип `ratio<>`, поэтому статический член `type` выдает соответствующий тип. Например, результат следующего выражения равен `std::ratio<13, 21>` (вычисленный как $6/21+7/21$):

```
std::ratio_add<std::ratio<2, 7>, std::ratio<2, 6>>::type
```

Таблица 5.17. Операции класса `ratio<>`

Операция	Действие	Результат
<code>ratio_add</code>	Сокращенная сумма рациональных чисел	<code>ratio<></code>
<code>ratio_subtract</code>	Сокращенная разность рациональных чисел	<code>ratio<></code>
<code>ratio_multiply</code>	Сокращенное произведение рациональных чисел	<code>ratio<></code>
<code>ratio_divide</code>	Сокращенное частное рациональных чисел	<code>ratio<></code>
<code>ratio_equal</code>	Проверка ==	<code>true_type</code> или <code>false_type</code>
<code>ratio_not_equal</code>	Проверка !=	<code>true_type</code> или <code>false_type</code>
<code>ratio_less</code>	Проверка <	<code>true_type</code> или <code>false_type</code>
<code>ratio_less_equal</code>	Проверка <=	<code>true_type</code> или <code>false_type</code>
<code>ratio_greater</code>	Проверка >	<code>true_type</code> или <code>false_type</code>
<code>ratio_greater_equal</code>	Проверка >=	<code>true_type</code> или <code>false_type</code>

Кроме того, можно сравнивать два рациональных типа с помощью предикатов `ratio_equal`, `ratio_not_equal`, `ratio_less`, `ratio_less_equal`, `ratio_greater` или `ratio_greater_equal`. Как и свойства типов, тип результата порождается из типов `true_type` или `false_type` (см. раздел 5.4.2), поэтому их член `value` возвращает значение `true` или `false`.

```
ratio_equal<ratio<5, 3>, ratio<25, 15>>::value // возвращает значение true
```

Как было сказано выше, класс `ratio` перехватывает все ошибки, возникающие на этапе компиляции, такие как деление на нуль или переполнение. Например, выражение

```
ratio_multiply<ratio<1, numeric_limits<long long>::max()>,
               ratio<1, 2>>::type
```


не скомпилируется, потому что умножение $1/\text{max}$ на $1/2$ приводит к переполнению и результирующее значение знаменателя выходит за пределы, допустимые для данного типа.

Аналогично не будет скомпилировано и следующее значение, поскольку оно приводит к делению на ноль:

```
ratio_divide<fiveThirds,zero>::type
```

Отметим, однако, что следующее выражение будет скомпилировано, потому что неверное значение обнаруживается только при вычислении членов `type`, `num` или `den`:

```
ratio_divide<fiveThirds,zero>
```

Таблица 5.18. Предопределенные значения класса `ratio`

Имя	Значение
yocto	1/ 1 000 000 000 000 000 000 000 000 (не обязательно)
zepto	1/1000 000 000 000 000 000 000 000 (не обязательно)
atto	1/1000 000 000 000 000 000
femto	1/1000 000 000 000 000
pico	1/1000 000 000 000
nano	1/1000 000 000
micro	1/1000 000
milli	1/1000
centi	1/100
deci	1/10
deca	10
hecto	100
kilo	1 000
mega	1 000 000
giga	1 000 000 000
tera	1 000 000 000 000
peta	1 000 000 000 000 000
exa	1 000 000 000 000 000 000
zetta	1 000 000 000 000 000 000 000 (не обязательно)
yotta	1 000 000 000 000 000 000 000 000 (не обязательно)

Предопределенные значения рациональных чисел позволяют удобно задавать очень большие или очень маленькие числа (см. табл. 5.18). Они позволяют задавать большие числа без утомительного перечисления нулей. Например, выражение

```
std::nano
```

эквивалентно выражению

```
std::ratio<1,1000000000LL>
```

С помощью этого значения можно задавать, например, наносекунды (см. раздел 5.7.2). Единицы, помеченные как необязательные, определены, только если их допускает тип `intmax_t`.

5.7. Часы и таймеры

Одной из самых необходимых библиотек в любом языке программирования является библиотека для работы с датами и временем. Однако опыт показывает, что такую библиотеку труднее разработать, чем это кажется. Проблема заключается в степени гибкости и точности, которую должна обеспечить эта библиотека. В прошлом интерфейсы к системному времени, предусмотренные в языке C и в стандарте POSIX, переключались с секунд на миллисекунды, затем на микросекунды и наконец на наносекунды. Проблема заключалась в том, что при каждом таком переключении требовался новый интерфейс. По этой причине в стандарте C++11 была предложена нейтральная в смысле точности библиотека. Эта библиотека обычно называется *хронобиблиотекой* (библиотекой Chrono), потому что она определена в заголовочном файле `<chrono>`.

Кроме того, стандартная библиотека языка C++ содержит базовые интерфейсы языка C и стандарта POSIX для работы с календарным временем. В заключение отметим, что библиотека потоков, предоставляемая стандартом C++11, позволяет приостановить выполнение потока или программы (главного потока) на определенный период времени.

5.7.1. Обзор библиотеки Chrono

Библиотека Chrono разработана так, чтобы учесть тот факт, что таймеры и часы в разных системах разные и со временем могут становиться все более точными. Для того чтобы избежать введения нового типа для времени, скажем, через десять лет, как это произошло с библиотеками времени в стандарте POSIX, разработчики постарались реализовать концепцию нейтральной точности, отделив понятие интервала и момента времени (“*timepoint*”) от конкретных часов. В результате ядро библиотеки Chrono состоит из следующих типов и понятий, служащих абстрактными механизмами для указания и обработки моментов и интервалов.

- *Интервал времени* (*duration*) определен как некоторое количество тактов определенной продолжительности (единицы времени). Например, интервал продолжительностью 3 минуты означает три такта по минуте. Другие примеры — “42 миллисекунды” или “86 400 секунд”, представляющий интервал, равный одним суткам. Эта концепция позволяет также задавать такие периоды, как “1,5 раза по трети секунды”, где 1,5 — количество тактов, а “треть секунды” — используемая единица времени.
- *Момент времени* (*timepoint*) определен как комбинация интервала и начала отсчета времени (так называемой эпохи). Типичным примером момента времени является “Полночь нового 2000-го года”, описываемая как “1 262 300 400 секунд с 1 января 1970 года” (этот день является эпохой (*epoch*) для системных часов в системах UNIX и POSIX).
- Однако концепция момента времени параметризована *часами* (*clock*), представляющими собой объект, определяющий эпоху момента времени. Таким образом, разные часы имеют разные эпохи. Операторы, обрабатывающие несколько моментов времени, например, вычисляющие интервал между двумя моментами времени (их

разность), требуют использования одной и той же эпохи (часов). Кроме того, часы предоставляют удобную функцию для выдачи *текущего* момента времени.

Иначе говоря, момент времени определен как интервал времени, прошедшего после эпохи, заданной часами (см. рис. 5.4).

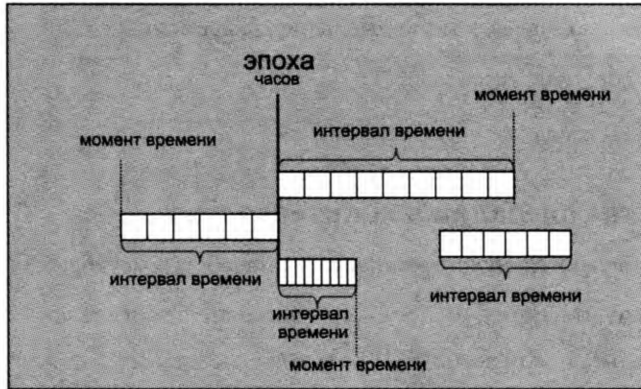


Рис. 5.4. Эпоха, интервалы и моменты времени

Более подробно о мотивации и разработке этих классов можно прочитать в работе [N2661:Chrono]²⁸. Рассмотрим подробнее эти типы и концепции.

Отметим, что все идентификаторы в библиотеке Chrono определены в пространстве имен `std::chrono`.

5.7.2. Интервалы времени

Интервал времени — это комбинация значения, представляющего количество тактов, и дроби, представляющей единицу времени в секундах. Для задания этой дроби используется класс `ratio` (см. раздел 5.6). Рассмотрим пример:

```
std::chrono::duration<int> twentySeconds(20);
std::chrono::duration<double, std::ratio<60>> halfAMinute(0.5);
std::chrono::duration<long, std::ratio<1,1000>> oneMillisecond(1);
```

где первый шаблонный аргумент определяет тип тактов, а необязательный второй шаблонный аргумент определяет тип единицы в секундах. Таким образом, первая строка использует в качестве типа единиц секунды, вторая — минуты (“60/1 секунд”), третья — миллисекунды (“1/1000 секунд”).

Для удобства стандартная библиотека языка C++ содержит следующие определения типов:

```
namespace std {
    namespace chrono {
        typedef duration<signed int-type >= 64 битов, nano> nanoseconds;
        typedef duration<signed int-type >= 55 битов, micro> microseconds;
```

²⁸ Цитаты из работы [N2661:Chrono] приводятся с любезного разрешения авторов.

```

typedef duration<signed int-type >= 45 битов, milli> milliseconds;
typedef duration<signed int-type >= 35 битов> seconds;
typedef duration<signed int-type >= 29 битов, ratio<60>> minutes;
typedef duration<signed int-type >= 23 битов, ratio<3600>> hours;
}
}

```

С их помощью легко задавать типичные периоды времени.

```

std::chrono::seconds twentySeconds(20);
std::chrono::hours aDay(24);
std::chrono::milliseconds oneMillisecond(1);

```

Арифметические операции над интервалами

Интервалы вычисляются вполне ожидаемым способом (см. табл. 5.19).

- Можно вычислить сумму, разность, произведение и частное двух интервалов.
- Можно складывать или вычитать такты и другие интервалы.
- Можно сравнивать два интервала.

Важным обстоятельством является то, что типы единицы для двух интервалов в таких операциях могут быть разными. Благодаря перегрузке класса `common_type<>` (см. раздел 5.4.1) для объектов класса `duration`, результирующий интервал будет измеряться в единицах измерения наибольшего общего знаменателя единиц обоих операндов. Например, после выполнения операторов

```

chrono::seconds d1(42); // 42 секунды
chrono::milliseconds d2(10); // 10 миллисекунды

```

выражение

```
d1 - d2
```

дает интервал, длина которого равна 41 990 тактов в миллисекундах (1/1000 секунды).

В более общем случае после выполнения операторов

```

chrono::duration<int, ratio<1,3>> d1(1); // 1 такт по 1/3 секунды
chrono::duration<int, ratio<1,5>> d2(1); // 1 такт по 1/5 секунды

```

выражение

```
d1 + d2
```

создаст интервал, состоящий из 8 тактов по 1/15, а выражение

```
d1 < d2
```

будет иметь значение `false`. В обоих случаях интервал `d1` состоит из 5 тактов по 1/15 секунд, а `d2` — из 3 тактов по 1/15 секунд. Таким образом, сумма 3 и 5 равна 8, а 5 больше 3.

Таблица 5.19. Арифметические операции над объектами класса `duration`

Операция	Действие
<code>d1 + d2</code>	Вычисляет сумму двух интервалов <code>d1</code> и <code>d2</code>
<code>d1 - d2</code>	Вычисляет разность между интервалами <code>d1</code> и <code>d2</code>
<code>d * val</code>	Возвращает результат, равный произведению <code>val</code> на интервал <code>d</code>
<code>val * d</code>	Возвращает результат, равный произведению <code>val</code> на интервал <code>d</code>
<code>d / val</code>	Возвращает интервал <code>d</code> , деленный на значение <code>val</code>
<code>d1 / d2</code>	Вычисляет частное от деления интервала <code>d1</code> на интервал <code>d2</code>
<code>d % val</code>	Вычисляет остаток от деления интервала <code>d1</code> на значение <code>val</code>
<code>d % d2</code>	Вычисляет остаток от деления интервала <code>d</code> на интервал <code>d2</code>
<code>d1 == d2</code>	Возвращает результат проверки того, что интервал <code>d1</code> равен интервалу <code>d2</code>
<code>d1 != d2</code>	Возвращает результат проверки того, что интервал <code>d1</code> не равен интервалу <code>d2</code>
<code>d1 < d2</code>	Возвращает результат проверки того, что интервал <code>d1</code> короче интервала <code>d2</code>
<code>d1 <= d2</code>	Возвращает результат проверки того, что интервал <code>d1</code> не длиннее интервала <code>d2</code>
<code>d1 > d2</code>	Возвращает результат проверки того, что интервал <code>d1</code> длиннее интервала <code>d2</code>
<code>d1 <= d2</code>	Возвращает результат проверки того, что интервал <code>d1</code> не короче интервала <code>d2</code>
<code>++d</code>	Увеличивает интервал <code>d</code> на 1 такт
<code>d++</code>	Увеличивает интервал <code>d</code> на 1 такт
<code>--d</code>	Уменьшает интервал <code>d</code> на 1 такт
<code>d--</code>	Уменьшает интервал <code>d</code> на 1 такт
<code>d += d1</code>	Увеличивает интервал <code>d</code> на длину интервала <code>d1</code>
<code>d -= d1</code>	Уменьшает интервал <code>d</code> на длину интервала <code>d1</code>
<code>d *= val</code>	Умножает интервал <code>d</code> на значение <code>val</code>
<code>d /= val</code>	Делит интервал <code>d</code> на значение <code>val</code>
<code>d %= val</code>	Делит интервал <code>d</code> на значение <code>val</code> по модулю
<code>d %= d2</code>	Делит интервал <code>d</code> на интервал <code>d2</code> по модулю

Можно также преобразовать интервалы из одних единиц измерения в другие, поскольку в классе предусмотрено неявное преобразование типа. Таким образом, можно преобразовать часы в секунды, но не наоборот. Рассмотрим примеры:

```
std::chrono::seconds twentySeconds(20); // 20 секунд
std::chrono::hours aDay(24);           // 24 часа
std::chrono::milliseconds ms(0);        // 0 миллисекунд
                                           // (без нуля значение не определено)
ms += twentySeconds + aDay;             // 86 420 000 миллисекунд
--ms;                                   // 86 419 999 миллисекунд
ms *= 2;                                 // 172 839 998 миллисекунд
std::cout << ms.count() << " ms" << std::endl;
std::cout << std::chrono::nanoseconds(ms).count() << " ns" << std::endl;
```

Эти преобразования приводят к таким результатам:

```
172839998 ms
172839998000000 ns
```

Другие операции над интервалами

В предыдущем примере мы использовали функцию-член `count()` для вычисления текущего количества тактов — одну из многих операций над интервалами. В табл. 5.18 приведены все операции, члены и типы, доступные для интервалов, помимо арифметических операций, указанных в табл. 5.20. Отметим, что конструктор по умолчанию выполняет инициализацию интервала значением, предусмотренным по умолчанию (см. раздел 3.2.1), т.е. для элементарных типов начальное значение не определено.

Таблица 5.20. Другие операции и типы в классе `duration`

Операция	Действие
<code>duration d</code>	Конструктор по умолчанию; создает интервал (инициализированный по умолчанию)
<code>duration d(d2)</code>	Копирующий конструктор (интервал <code>d2</code> может иметь разные типы измерения)
<code>duration d(val)</code>	Создает интервал, состоящий из <code>val</code> тактов, имеющих тип, предусмотренный для интервала <code>d</code>
<code>d = d2</code>	Присваивает интервал <code>d2</code> интервалу <code>d</code> (возможно неявное преобразование)
<code>d.count()</code>	Возвращает количество тактов в интервале <code>d</code>
<code>duration_cast<D>(d)</code>	Возвращает интервал <code>d</code> , явно преобразованный в тип <code>D</code>
<code>duration::zero()</code>	Создает интервал нулевой длины
<code>duration::max()</code>	Создает максимально возможный интервал данного типа
<code>duration::min()</code>	Создает минимально возможный интервал данного типа
<code>duration::rep</code>	Возвращает тип тактов
<code>duration::period</code>	Возвращает тип единицы измерения

Эти члены можно использовать для определения удобной функции для оператора вывода `<< интервалов`²⁹.

```
template <typename V, typename R>
ostream& operator << (ostream& s, const chrono::duration<V,R>& d)
{
    s << "[" << d.count() << " of " << R::num << "/"
      << R::den << "];"
    return s;
}
```

²⁹ Отметим, что этот оператор вывода не работает без механизма *ADL* (*argument-dependent lookup* — поиск с учетом аргументов) (см. раздел 15.11.1).

Здесь после вывода количества тактов с помощью функции `count()` мы выводим числитель и знаменатель используемой единицы измерения, которую класс `ratio` создает на этапе компиляции (см. раздел 5.6). Например, операторы

```
std::chrono::milliseconds d(42);
std::cout << d << std::endl;
```

выводят на экран строку

```
[42 of 1/1000]
```

Как видим, всегда существует возможность неявного преобразования в более точный тип единицы измерения. Однако неявное преобразование в менее точный тип единицы измерения невозможно, поскольку при этом происходит потеря информации. Например, при преобразовании целочисленного значения 42 010 миллисекунд в секунды результирующее целочисленное значение 42 означает потерю 10 миллисекунд. Однако такое преобразование можно выполнить принудительно с помощью оператора `duration_cast`. Например:

```
std::chrono::seconds sec(55);
std::chrono::minutes m1 = sec; // ОШИБКА
std::chrono::minutes m2 =
    std::chrono::duration_cast<std::chrono::minutes>(sec); // OK
```

В качестве другого примера отметим, что преобразование интервала с тактами, имеющими тип числа с плавающей точкой, также требует явного приведения при преобразовании в целочисленный интервал.

```
std::chrono::duration<double, std::ratio<60>> halfMin(0.5);
std::chrono::seconds s1 = halfMin; // ОШИБКА
std::chrono::seconds s2 =
    std::chrono::duration_cast<std::chrono::seconds>(halfMin); // OK
```

Типичный пример — код, разделяющий интервал на разные единицы. Например, следующий код разделяет интервал, состоящий из миллисекунд, на соответствующее количество часов, минут, секунд и миллисекунд (первая строка вывода начинается с выражения `raw:` и является результатом выполнения оператора вывода, которую мы только что определили).

```
using namespace std;
using namespace std::chrono;
milliseconds ms(7255042);

// разделяем интервал на часы, минуты, секунды и миллисекунды
hours hh = duration_cast<hours>(ms);
minutes mm = duration_cast<minutes>(ms % chrono::hours(1));
seconds ss = duration_cast<seconds>(ms % chrono::minutes(1));
milliseconds msec
    = duration_cast<milliseconds>(ms % chrono::seconds(1));

// выводим интервалы и значения:
cout << "raw: " << hh << ":" << mm << ":" << ss << ":" << msec;
```

```

        << ss << ":" << msec << endl;
cout << " " << setfill('0') << setw(2) << hh.count() << ":" <<
        << setw(2) << mm.count() << ":" <<
        << setw(2) << ss.count() << ":" <<
        << setw(3) << msec.count() << endl;

```

Здесь приведение

```
std::chrono::duration_cast<std::chrono::hours>(ms)
```

преобразовывает миллисекунды в часы, усекая, а не округляя значение. Благодаря оператору деления по модулю %, которому в качестве второго операнда можно передавать интервал, можно легко обработать оставшиеся миллисекунды с помощью оператора `ms % std::chrono::hours(1)`, которые затем преобразовываются в минуты. Таким образом, вывод этого кода выглядит следующим образом:

```
raw: [2 of 3600/1]::[0 of 60/1]::[55 of 1/1]::[42 of 1/1000]
    02::00::55::042
```

В заключение отметим, что класс `duration` содержит три статические функции: `zero()`, создающую интервал нулевой длины в секундах, а также `min()` и `max()`, создающие минимально и максимально возможные интервалы.

5.7.3. Часы и моменты времени

Отношения между моментами времени и часами довольно запутанные.

- *Часы* определяют эпоху и продолжительность такта. Например, часы могут иметь такт, измеряемый в миллисекундах, начиная с эпохи UNIX (1 января 1970 года), или в наносекундах, начиная с запуска программы. Кроме того, часы задают тип любого момента времени, определяемого по данным часам.
- Интерфейс часов содержит функцию `now()`, возвращающую объект для текущего момента времени.
- *Момент времени* представляет конкретный момент времени, связанный с положительным или отрицательным интервалом, измеренным по заданным часам. Таким образом, интервал “10 суток” и ассоциированные с ним часы с эпохой “1 января 1970 года” соответствует 11 января 1970 года.
- Интерфейс момента времени позволяет определить эпоху, минимальный и максимальный моменты времени, возможные для данных часов, а также арифметические операции над моментами времени.

Часы

В табл. 5.21 приведены определения типов и статические члены, требуемые для всех часов.

Таблица 5.21. Операции и типы часов

Операция	Действие
<code>часы::duration</code>	Возвращает тип интервала для часов
<code>часы::rep</code>	Возвращает тип такта (эквивалент <code>часы::duration::rep</code>)
<code>часы::period</code>	Возвращает тип единицы измерения (эквивалент <code>часы::duration::period</code>)
<code>часы::time_point</code>	Возвращает тип момента времени для часов
<code>часы::is_steady</code>	Возвращает значение <code>true</code> , если часы являются монотонными
<code>часы::now()</code>	Возвращает объект класса <code>time_point</code> для текущего момента времени

Стандартная библиотека языка C++ содержит три вида часов, имеющих собственный интерфейс.

1. Класс `system_clock` представляет моменты времени, связанные с обычными часами реального времени в текущей операционной системе. Эти часы также имеют вспомогательные функции `to_time_t()` и `from_time_t()`, чтобы конвертировать любой момент времени в объект типа `time_t`, предусмотренного в языке C, и наоборот. Это означает, что его можно конвертировать в календарную дату, и наоборот (см. раздел 5.7.4).
2. Класс `steady_clock` предоставляет гарантии того, что часы никогда не будут корректироваться³⁰. Таким образом, значения моментов времени, измеряемых по этим часам, никогда не уменьшаются и возрастают равномерно по отношению к реальному времени.
3. Класс `high_resolution_clock` представляет часы с наименьшей продолжительностью такта, возможной в текущей системе.

Отметим, что стандарт не формулирует требований к точности, эпохе и диапазону (минимальному и максимальному моментам времени) перечисленных часов. Например, ваши системные часы могут иметь эпоху UNIX (1 января 1970 года), но это ничем не гарантируется. Если вам потребуется конкретная эпоха или моменты времени, которых нет у перечисленных часов, придется воспользоваться вспомогательными функциями.

Например, следующая функция выводит на экран свойства часов:

```
// util/clock.hpp

#include <chrono>
#include <iostream>
#include <iomanip>

template <typename C>
void printClockData ()
{
    using namespace std;

    cout << "- precision: ";
    // если момент времени меньше или равен одной миллисекунде
```

³⁰ Класс `steady_clock` сначала предполагалось назвать `monotonic_clock`.

```
typedef typename C::period P; // Тип единицы времени
if (ratio_less_equal<P,milli>::value) {
    // преобразовываем в миллисекунды и выводим на экран
    typedef typename ratio_multiply<P,kilo>::type TT;
    cout << fixed << double(TT::num)/TT::den
        << " milliseconds" << endl;
}
else {
    // выводим на экран в виде секунд
    cout << fixed << double(P::num)/P::den << " seconds" << endl;
}
cout << "- is_steady: " << boolalpha << C::is_steady << endl;
}
```

Эту функцию можно вызвать для разных часов, предусмотренных в стандартной библиотеке языка C++.

```
// util/clock1.cpp

#include <chrono>
#include "clock.hpp"

int main()

{
    std::cout << "system_clock: " << std::endl;
    printClockData<std::chrono::system_clock>();
    std::cout << "\nhigh_resolution_clock: " << std::endl;
    printClockData<std::chrono::high_resolution_clock>();
    std::cout << "\nsteady_clock: " << std::endl;
    printClockData<std::chrono::steady_clock>();
}
```

Эта программа может вывести, например, следующие строки:

```
system_clock:
- precision: 0.000100 milliseconds
- is_steady: false

high_resolution_clock:
- precision: 0.000100 milliseconds
- is_steady: true

steady_clock:
- precision: 1.000000 milliseconds
- is_steady: true
```

Здесь, например, системные часы или часы высокого разрешения имеют одинаковую точность, равную 100 наносекундам, в то время как монотонные часы используют миллисекунды. Кроме того, монотонные часы и часы высокого разрешения невозможно корректировать. Отметим, однако, что в разных системах могут быть совершенно разные ситуации. Например, часы высокого разрешения могут не отличаться от системных часов.

Класс `steady_clock` играет важную роль при сравнении разности между двумя моментами времени в программе, когда вы обрабатываете текущий момент времени. Например, после выполнения оператора

```
auto system_start = chrono::system_clock::now();
```

проверка, работает ли программа дольше одной минуты,

```
if (chrono::system_clock::now() > system_start + minutes(1))
```

может не сработать, потому что, если в это время происходит коррекция часов, оператор сравнения может вернуть значение `false`, даже если на самом деле программа работала дольше минуты. Аналогично вычисление времени работы программы

```
auto diff = chrono::system_clock::now() - system_start;
auto sec = chrono::duration_cast<chrono::seconds>(diff);
cout << "this program runs: " << s.count() << " seconds" << endl;
```

может привести к отрицательному интервалу времени, если за время проверки часы корректировались. По этой же причине использование таймеров, отличающихся от класса `steady_clock`, может привести к изменению интервала времени из-за коррекции системных часов (см. раздел 5.7.5).

Моменты времени

При работе с любыми часами — даже с часами, заданными пользователем, — можно обрабатывать моменты времени. Соответствующий интерфейс, параметризованный типом часов, предоставляет класс `time_point`.

```
namespace std {
    namespace chrono {
        template <typename Clock,
                 typename Duration = typename Clock::duration>
        class time_point;
    }
}
```

Особую роль играют конкретные моменты времени.

1. *Эпоха*, которую для каждого часов создает конструктор по умолчанию класса `time_point`.
2. *Текущее время*, которое для каждого часов выдает статическая функция-член `now()` (см. раздел 5.7.3).
3. *Минимальный момент времени*, который для каждого часов выдает статическая функция-член `min()` класса `time_point`.
4. *Максимальный момент времени*, который для каждого часов выдает статическая функция-член `max()` класса `time_point`.

Например, следующая программа присваивает эти моменты времени объекту `tp` и выводит на экран в календарных обозначениях:

```
// util/chronol.cpp

#include <chrono>
#include <ctime>
#include <string>
#include <iostream>

std::string asString (const std::chrono::system_clock::time_point& tp)
{
    // преобразование в системное время:
    std::time_t t = std::chrono::system_clock::to_time_t(tp);
    std::string ts = std::ctime(&t); // преобразование в календарное время
    ts.resize(ts.size()-1);          // пропуск символа новой строки
    return ts;
}

int main()
{
    // вывод на экран эпохи для системных часов:
    std::chrono::system_clock::time_point tp;
    std::cout << "epoch: " << asString(tp) << std::endl;

    // вывод текущего времени:
    tp = std::chrono::system_clock::now();
    std::cout << "now: " << asString(tp) << std::endl;

    // вывод минимального времени для системных часов:
    tp = std::chrono::system_clock::time_point::min();
    std::cout << "min: " << asString(tp) << std::endl;

    // вывод максимального времени для системных часов:
    tp = std::chrono::system_clock::time_point::max();
    std::cout << "max: " << asString(tp) << std::endl;
}
```

После включения заголовочного файла `<chrono>` мы сначала объявляем вспомогательную функцию `asString()`, которая преобразовывает момент времени по системным часам в соответствующее календарное время. В коде

```
std::time_t t = std::chrono::system_clock::to_time_t(tp);
```

используется статическая вспомогательная функция `to_time_t()`, преобразовывающая момент времени в объект традиционного типа времени `time_t`, принятого в языке C и стандарте POSIX, который обычно представляет собой количество секунд, прошедших с эпохи UNIX, т.е. 1 января 1970 года (см. раздел 5.7.4). Затем оператор `std::string ts = std::ctime(&t);` с помощью функции `ctime()` преобразовывает его в календарное время, из которого инструкция `ts.resize(ts.size()-1);` удаляет пустую строку.

Отметим, что функция `ctime()` учитывает часовой пояс. Последствия этого факта мы вскоре обсудим. Кроме того, как правило, эта вспомогательная функция работает только с классом `system_clock`, единственным классом, имеющим интерфейс для преобразования в тип `time_t`, и наоборот. Для других часов такой интерфейс может работать, но не быть переносимым, поскольку другие часы не требуют использования эпохи системного времени в качестве внутренней эпохи.

Отметим также, что формат вывода моментов времени иногда целесообразно локализовать с помощью аспекта `time_put`. Детали и примеры описаны в разделе 16.4.3.

В функции `main()` тип объекта `tp` объявлен как

```
std::chrono::system_clock::time_point
```

что эквивалентно³¹

```
std::chrono::time_point<std::chrono::system_clock>
```

Таким образом, объект `tp` объявлен как момент времени по часам типа `system_clock`. Возможность задавать часы как шаблонный аргумент гарантирует, что арифметика моментов времени возможна только с одними и теми же часами (эпохой).

Эта программа может вывести следующий результат:

```
epoch: Thu Jan  1 01:00:00 1970
now:   Sun Jul 24 19:40:46 2011
min:   Sat Mar  5 18:27:38 1904
max:   Mon Oct 29 07:32:22 2035
```

Таким образом, конструктор по умолчанию, определяющий эпоху, создает момент времени, который функция `asString()` преобразовывает в строку

```
Thu Jan 1 01:00:00 1970
```

Отметим, что этот момент времени соответствует часу ночи, а не полуночи. Это может показаться удивительным, но вспомним, что преобразование в календарное время с помощью функции `ctime()` в функции `asString()` учитывает часовой пояс.

Таким образом, эпоха UNIX, использованная здесь, — которая, напомним, не всегда совпадает с эпохой системного времени, — началась в 00:00 по Гринвичу. В моем часовом поясе, в Германии, в этот момент был час ночи, поэтому здесь эпоха началась в час ночи 1 января 1970 года. Соответственно, если вы запустите эту программу у себя на компьютере, то можете получить другой вывод, потому что он зависит от вашего часового пояса, даже если ваша система использует ту же самую эпоху, что и системные часы.

Для работы с универсальным временем (UTC) вместо вызова функции `ctime()` (которая является всего лишь сокращением для вызова `asctime(localtime(...))` (см. раздел 5.7.4)) следует использовать следующее преобразование:

```
std::string ts = std::asctime(gmtime(&t));
```

В данном случае вывод программы может оказаться таким:

```
epoch: Thu Jan  1 00:00:00 1970
now:   Sun Jul 24 17:40:46 2011
min:   Sat Mar  5 17:27:38 1904
max:   Mon Oct 29 06:32:22 2035
```

Да, здесь функция `now()` выдает разницу, равную двум часам, потому что этот момент времени соответствует летнему времени, которое в часовом поясе Германии создает двухчасовую разницу с универсальным временем.

³¹ В соответствии со стандартом класс `system_clock::time_point` может быть также идентичным классу `time_point<C2, system_clock::duration>`, где `C2` — другие часы, имеющие ту же самую эпоху, что и класс `system_clock`.

В принципе, объекты класса `time_point` должны иметь только один член, интервал, соответствующий эпохе связанных с ними часов. Этот момент времени можно запросить с помощью функции `time_since_epoch()`. Для выполнения арифметических операций над моментами времени предусмотрены полезные комбинации моментов времени и интервалов (табл. 5.22).

Таблица 5.22. Операции в классе `time_point`

Операция	Результат	Действие
<code>момент_времени t</code>	<code>момент времени</code>	Конструктор по умолчанию; создает момент времени, представляющий эпоху
<code>момент_времени t(tp2)</code>	<code>момент времени</code>	Создает момент времени, эквивалентный объекту <code>tp2</code> (единица измерения интервала может уменьшаться)
<code>момент_времени t(d)</code>	<code>момент времени</code>	Создает момент времени, соответствующий интервалу <code>d</code> после эпохи
<code>time_point_cast<C,D>(tp)</code>	<code>момент времени</code>	Преобразовывает объект <code>tp</code> в момент времени по часам <code>C</code> , соответствующий интервалу <code>D</code> (единица измерения интервала может увеличиваться)
<code>tp += d</code>	<code>момент времени</code>	Добавляет интервал <code>d</code> к текущему моменту времени <code>tp</code>
<code>tp -= d</code>	<code>момент времени</code>	Вычитает интервал <code>d</code> из текущего момента времени <code>tp</code>
<code>tp + d</code>	<code>момент времени</code>	Возвращает новый момент времени <code>tp</code> с добавленным интервалом <code>d</code>
<code>d + tp</code>	<code>момент времени</code>	Возвращает новый момент времени <code>tp</code> с добавленным интервалом <code>d</code>
<code>tp - d</code>	<code>момент времени</code>	Возвращает новый момент времени <code>tp</code> с вычтенным интервалом <code>d</code>
<code>tp1 - tp2</code>	<code>момент времени</code>	Возвращает интервал между моментами времени <code>tp1</code> и <code>tp2</code>
<code>tp1 == tp2</code>	<code>bool</code>	Возвращает результат проверки, равен ли момент времени <code>tp1</code> моменту времени <code>tp2</code>
<code>tp1 != tp2</code>	<code>bool</code>	Возвращает результат проверки, отличается ли момент времени <code>tp1</code> от момента времени <code>tp2</code>
<code>tp1 < tp2</code>	<code>bool</code>	Возвращает результат проверки, наступил ли момент времени <code>tp1</code> до момента времени <code>tp2</code>
<code>tp1 <= tp2</code>	<code>bool</code>	Возвращает результат проверки, не наступил ли момент времени <code>tp1</code> не позже момента времени <code>tp1</code>
<code>tp1 > tp2</code>	<code>bool</code>	Возвращает результат проверки, наступил ли момент времени <code>tp1</code> после момента времени <code>tp2</code>

Окончание табл. 5.22

Операция	Результат	Действие
<i>tp1</i> >= <i>tp2</i>	bool	Возвращает результат проверки, не наступил ли момент времени <i>tp1</i> не ранее момента времени <i>tp1</i>
<i>tp.time_since_epoch()</i>	интервал	Возвращает интервал между эпохой и моментом времени <i>tp</i>
<i>момент_времени::min()</i>	момент времени	Возвращает первый возможный момент времени типа <i>момент_времени</i>
<i>момент_времени::max()</i>	момент времени	Возвращает последний возможный момент времени типа <i>момент_времени</i>

Несмотря на то что интерфейс использует класс `ratio` (см. раздел 5.6), гарантирующий, что переполнение при вычислении интервала сгенерирует ошибку на этапе компиляции, переполнение при работе с интервалами все же возможно.

Рассмотрим следующий пример:

```
// util/chrono2.cpp

#include <chrono>
#include <ctime>
#include <iostream>
#include <string>
using namespace std;

string asString (const chrono::system_clock::time_point& tp)
{
    time_t t = chrono::system_clock::to_time_t(tp); // преобразуем
                                                    // в системное время
    string ts = ctime(&t); // преобразуем в календарное время
    ts.resize(ts.size()-1); // удаляем завершающий символ новой строки
    return ts;
}

int main()
{
    // определяем тип интервала для представления дней:
    typedef chrono::duration<int, ratio<3600*24>> Days;

    // определяем эпоху системных часов
    chrono::time_point<chrono::system_clock> tp;
    cout << "epoch: " << asString(tp) << endl;

    // добавляем одни сутки, 23 часа и 55 минут
    tp += Days(1) + chrono::hours(23) + chrono::minutes(55);
    cout << "later: " << asString(tp) << endl;

    // вычисляем длительность до эпохи в минутах и сутках:
    auto diff = tp - chrono::system_clock::time_point();
    cout << "diff: "
        << chrono::duration_cast<chrono::minutes>(diff).count()
        << " minute(s)" << endl;
    Days days = chrono::duration_cast<Days>(diff);
```

```
cout << "diff: " << days.count() << " day(s)" << endl;

// вычитаем один год (надеясь, что результат будет корректным
// и год не високосный)
tp -= chrono::hours(24*365);
cout << "-1 year: " << asString(tp) << endl;

// вычитаем 50 лет (надеясь, что результат будет корректным
// и игнорируя високосные годы)
tp -= chrono::duration<int, ratio<3600*24*365>>(50);
cout << "-50 years: " << asString(tp) << endl;

// вычитаем 50 лет (надеясь, что результат будет корректным
// и игнорируя високосные годы)
tp -= chrono::duration<int, ratio<3600*24*365>>(50);
cout << "-50 years: " << asString(tp) << endl;
}
```

Во-первых, выражения, такие как

```
tp = tp + Days(1) + chrono::hours(23) + chrono::minutes(55);
```

или

```
tp -= chrono::hours(24*365);
```

позволяют корректировать моменты времени с помощью арифметических операций над моментами времени.

Поскольку точность системных часов обычно выше, чем минуты и сутки, необходимо явно привести разницу между двумя моментами времени к суткам.

```
auto diff = tp - chrono::system_clock::time_point();
Days days = chrono::duration_cast<Days>(diff);
```

Однако следует подчеркнуть, что эти операции не проверяют возможность переполнения. На нашем компьютере результат работы программы был следующим:

```
epoch:      Thu Jan  1 01:00:00 1970
later:      Sat Jan  3 00:55:00 1970
diff:       2875 minute(s)
diff:       1 day(s)
-1 year:    Fri Jan  3 00:55:00 1969
-50 years:  Thu Jan 16 00:55:00 1919
-50 years:  Sat Mar  5 07:23:16 2005
```

Итак, можно сделать соответствующие выводы.

- Приведение к требуемой единице измерения, которая обычно имеет целочисленный тип, использует оператор `static_cast<>`, а это означает округление. По этой причине интервал, состоящий из 47 часов и 55 минут, после преобразования оказывается равным одним суткам.
- Вычитание 50 лет, состоящих из 365 суток, не учитывает високосные годы, поэтому результат равен 16 января, а не 3 января.

- При повторном вычитании 50 лет момент времени оказывается меньше минимального, который на нашем компьютере соответствует 5 марта 1904 года (см. раздел 5.7.3), поэтому результат равен 2005. В данном случае никаких ошибок нет.

Этот пример демонстрирует, что библиотека `Chrono` является библиотекой интервалов и моментов времени, а не дат и времени. Вы можете вычислять интервалы и моменты времени, но должны учитывать эпоху, минимальный и максимальный моменты времени, високосные годы и секунды координации.

5.7.4. Функции для работы с датами и временем в языке C и стандарте POSIX

В стандартной библиотеке языка C++ есть также стандартные интерфейсы C и POSIX для работы с датами и временем. Макросы и типы из заголовка `<ctime>`, а также функции из заголовочного файла `<time.h>` доступны в пространстве имен `std`. Типы и функции для работы с датами и временем перечислены в табл. 5.23. Кроме того, макрос `CLOCKS_PER_SEC` определяет тип единицы измерения для функции `clock()` (которая возвращает прошедшее время работы центрального процессора, измеренное в $1/\text{CLOCKS_PER_SEC}$ долях секунд). Подробности и примеры, касающиеся функций и типов для работы с датами и временем, описаны в разделе 16.4.3.

Таблица 5.23. Определения в заголовочном файле `<ctime>`

Идентификатор	Смысл
<code>clock_t</code>	Тип числовых величин для измерения прошедшего времени работы центрального процессора, возвращаемых функцией <code>clock()</code>
<code>time_t</code>	Тип числовых величин, представляющих моменты времени
<code>struct tm</code>	Тип сокращенного календарного времени
<code>clock()</code>	Возвращает прошедшее время работы центрального процессора, измеренное в $1/\text{CLOCKS_PER_SEC}$ долях секунды
<code>time()</code>	Возвращает текущее время как числовую величину
<code>difftime()</code>	Возвращает разницу между двумя моментами времени типа <code>time_t</code> , измеренную в секундах и представленную как число типа <code>double</code>
<code>localtime()</code>	Преобразовывает объект типа <code>time_t</code> в <code>struct tm</code> с учетом часового пояса
<code>gmtime()</code>	Преобразовывает объект типа <code>time_t</code> в <code>struct tm</code> без учета часового пояса
<code>asctime()</code>	Преобразовывает объект типа <code>struct tm</code> в стандартную строку, представляющую календарное время
<code>strftime()</code>	Преобразовывает объект типа <code>struct tm</code> в строку, представляющую календарное время и определенную пользователем
<code>ctime()</code>	Преобразовывает объект типа <code>time_t</code> в стандартную строку, представляющую календарное время с учетом часового пояса (сокращение для вызова функции <code>asctime(localtime(t))</code>)
<code>mktime()</code>	Преобразовывает объект типа <code>struct tm</code> в объект типа <code>time_t</code> и вычисляет день недели и год

Отметим, что тип `time_t` обычно выражает количество секунд, прошедших с эпохи UNIX, т.е. с 1 января 1970 года. Однако в соответствии со стандартами языков C и C++ это свойство не гарантируется.

Преобразования между моментами времени и календарным временем

Вспомогательная функция для преобразования момента времени в строку, представляющую календарное время, уже обсуждалась в разделе 5.7.3. Рассмотрим заголовочный файл, который также позволяет конвертировать календарное время в моменты времени.

```
// util/timepoint.hpp

#include <chrono>
#include <ctime>
#include <string>

// преобразование момента времени по системным часам
// в строку календарного времени
inline
std::string asString (const std::chrono::system_clock::time_point& tp)
{
    // преобразование в системное время:
    std::time_t t = std::chrono::system_clock::to_time_t(tp);
    std::string ts = ctime(&t); // преобразование в календарное время
    ts.resize(ts.size()-1);      // опускаем завершающий символ новой строки
    return ts;
}

// преобразование календарного времени
// в момент времени по системным часам
inline
std::chrono::system_clock::time_point
makeTimePoint (int year, int mon, int day,
int hour, int min, int sec=0)
{
    struct std::tm t;
    t.tm_sec = sec;           // секунд минуты (0 .. 59 и
                              // 60 для секунд координации)
    t.tm_min = min;          // минута часа (0 .. 59)
    t.tm_hour = hour;         // час суток (0 .. 23)
    t.tm_mday = day;          // день месяца (1 .. 31)
    t.tm_mon = mon-1;         // месяц года (0 .. 11)
    t.tm_year = year-1900;    // год, считая от 1900
    t.tm_isdst = -1;          // определить, учитывается ли летнее время
    std::time_t tt = std::mktime(&t);
    if (tt == -1) {
        throw "no valid system time";
    }
    return std::chrono::system_clock::from_time_t(tt);
}
```

Эти функции иллюстрируются следующей программой:

```
// util/timepoint1.cpp
#include <chrono>
#include <iostream>
#include "timepoint.hpp"

int main()
{
    auto tp1 = makeTimePoint(2010, 01, 01, 00, 00);
    std::cout << asString(tp1) << std::endl;
    auto tp2 = makeTimePoint(2011, 05, 23, 13, 44);
    std::cout << asString(tp2) << std::endl;
}
```

Эта программа выдает следующие результаты:

```
Fri Jan 1 00:00:00 2010
Mon May 23 13:44:00 2011
```

Еще раз отметим, что функции `makeTimePoint()` и `asString()` учитывают часовой пояс. По этой причине дата, передаваемая функции `makeTimePoint()`, совпадает с результатом работы функции `asString()`. Кроме того, летнее время никакой роли не играет (передача отрицательного значения `t.tm_isdst` в функцию `makeTimePoint()` вынуждает функцию `mktime()` самостоятельно определить, учитывается ли летнее время при измерениях).

Для того чтобы функция `asString()` использовала не календарное, а универсальное время, следует использовать функцию `asctime(gmtime(...))`, а не `ctime(...)`. Для функции `mktime()` не существует механизма, заставляющего ее использовать время UTC, поэтому функция `makeTimePoint()` всегда учитывает часовой пояс.

Использование локальных установок для интернационализации чтения и записи показаний времени описано в разделе 16.4.3.

5.7.5. Блокировка с помощью таймеров

Интервалы и моменты времени можно использовать для блокировки потоков или программы (т.е. главного потока). Эти блоки могут быть безусловными, а могут использоваться для указания максимального интервала ожидания блокировки, переменной условия или завершения другого потока (см. главу 18).

- Функции `sleep_for()` и `sleep_until()` являются членами класса `this_thread` и предназначены для блокировки потоков (см. раздел 18.3.7).
- Функции `try_lock_for()` и `try_lock_until()` предназначены для задания максимального интервала ожидания для мьютекса (см. раздел 18.5.1).
- Функции `wait_for()` и `wait_until()` предназначены для задания максимального интервала при ожидании переменной условия или будущего момента времени (см. разделы 18.1.1 и 18.6.4).

Все блокирующие функции, имена которых завершаются суффиксом `_for()`, используют как аргумент интервал, а функции, имена которых завершаются суффиксом `_until()`, получают в качестве аргумента момент времени. Например, оператор

```
this_thread::sleep_for(chrono::seconds(10));
```

блокирует текущий поток, который может быть главным, на 10 секунд, а оператор

```
this_thread::sleep_until(chrono::system_clock::now()
+ chrono::seconds(10));
```

блокирует текущий поток, пока системные часы не достигнут отметки 10 секунд, считая от текущего момента.

Несмотря на то что все эти вызовы могут казаться одинаковыми, это не так! Для всех функций с суффиксом `_until()` при передаче аргумента допускается корректировка времени. Если на протяжении 10 секунд после вызова функции `sleep_until()` системные часы будут скорректированы, то задержка также будет уточнена соответствующим образом. Если, например, перевести системные часы назад на один час, то программа будет заблокирована на 60 минут и 10 секунд. Если же перевести часы вперед больше чем на 10 секунд, то таймер немедленно остановится.

При использовании функции с суффиксом `_for()`, например `sleep_for()`, получающей интервал, а также при использовании класса `steady_clock` корректировка системных часов, как правило, не влияет на продолжительность работы таймера. Однако если аппаратное обеспечение не имеет монотонных часов и поэтому нет возможности подсчитывать секунды независимо от потенциально скорректированного системного времени, то корректировки времени влияют также и на функции с суффиксом `_for()`.

Все таймеры могут быть неточными. Любой таймер может отставать, потому что система периодически проверяет просроченные таймеры, обрабатывает их и делает прерывания. Таким образом, интервалы таймеров могут равняться заданному периоду плюс периоду, который зависит от качества их реализации и текущей ситуации.

5.8. Заголовочные файлы `<cstdint>`, `<cstdlib>` и `<cstring>`

Следующие заголовочные файлы, совместимые с языком C, часто используются в программах на языке C++: `<cstdint>`, `<cstdlib>` и `<cstring>`. Они представляют собой версии заголовочных файлов `<stdint.h>`, `<stdlib.h>` и `<string.h>` из языка C, предназначенные для языка C++. В них определены некоторые общие константы, макросы, типы и функции.

5.8.1. Определения в заголовочном файле `<cstdint>`

В табл. 5.24 показаны определения из заголовочного файла `<cstdint>`. До появления стандарта C++11 значение `NULL` часто использовалось для описания ситуации, в которой указатель ни на что не ссылался. В стандарте C++11 эта семантика описывается значением `nullptr` (см. раздел 3.1.1).

Обратите внимание на то, что значение `NULL` в языке C++ гарантированно равно 0 (независимо от типа: `int` или `long`). В языке C значение `NULL` часто определяется как `(void*)0`. Однако в языке C++ это выражение является некорректным, потому что требуется, чтобы тип значения `NULL` был целочисленным. В противном случае значение `NULL` нельзя присвоить указателю. Это происходит потому, что в языке C++ нет автоматического преобразования типа `void*` в любой другой тип. В стандарте C++11 вместо `NULL`

следует использовать `nullptr` (см. раздел 3.1.1)³². Отметим также, что значение `NULL` также определено в заголовочных файлах <stdio>, <stdlib>, <string>, <ctime>, <wchar> и <locale>.

Таблица 5.24. Определения в заголовочном файле <cstdlib>

Идентификатор	Смысл
<code>NULL</code>	Значение пустого указателя
<code>nullptr_t</code>	Тип значения <code>nullptr</code> (по стандарту C++11)
<code>size_t</code>	Беззнаковый тип для единиц размера, например для количества элементов
<code>ptrdiff_t</code>	Знаковый тип для разности между указателями
<code>max_align_t</code>	Тип для максимального выравнивания во всех контекстах (по стандарту C++11)
<code>offsetof(<i>type</i>, <i>mem</i>)</code>	Смещение члена <i>mem</i> в структуре или объединении <i>type</i>

5.8.2. Определения в заголовочном файле <stdlib>

В табл. 5.25 приведены наиболее важные определения в заголовочном файле <stdlib>. Две константы, `EXIT_SUCCESS` и `EXIT_FAILURE`, определены как аргументы функции `exit()` и могут использоваться в качестве значения, возвращаемого функцией `main()`.

Таблица 5.25. Определения в заголовочном файле <stdlib>

Определение	Смысл
<code>EXIT_SUCCESS</code>	Нормальное завершение программы
<code>EXIT_FAILURE</code>	Ненормальное завершение программы
<code>exit(int <i>status</i>)</code>	Выход из программы (очистка статических объектов)
<code>quick_exit(int <i>status</i>)</code>	Выход из программы с очисткой, предусмотренной функцией <code>at_quick_exit()</code> (по стандарту C++11)
<code>_Exit(int <i>status</i>)</code>	Выход из программы без очистки (по стандарту C++11)
<code>abort()</code>	Аварийное завершение программы (может привести к краху в некоторых системах)
<code>atexit(void (*<i>func</i>)())</code>	Вызов функции <i>func</i> при выходе из программы
<code>at_quick_exit(void (*<i>func</i>)())</code>	Вызов функции <i>func</i> при выполнении функции <code>quick_exit()</code> (по стандарту C++11)

При нормальном завершении программы функции, зарегистрированные функцией `atexit()`, вызываются в порядке, обратном порядку их регистрации. При этом не имеет значения, завершается ли программа функцией `exit()` или достигнут конец функции `main()`. Аргументы функциям не передаются.

³² Из-за путаницы с типом значения `NULL` некоторые люди и руководства по стилю программирования рекомендуют вообще не использовать значение `NULL` в программах на языке C++. Вместо него лучше использовать 0 или специальную константу, определенную пользователем, например `NIL`. К счастью, концепция `nullptr` решила эту проблему.

Функции `exit()` и `abort()` прерывают выполнение программы в любой функции без возвращения в функцию `main()`.

- Функция `exit()` уничтожает все статические объекты, очищает все буфера, закрывает все каналы ввода-вывода и завершает выполнение программы, включая вызовы функций `atexit()`. Если функции, переданные функции `atexit()`, генерируют исключения, вызывается функция `terminate()`.
- Функция `abort()` немедленно прекращает выполнение программы без очистки.

Ни одна из этих функций не удаляет локальные объекты, потому что при их вызове не происходит раскручивание стека. Для того чтобы гарантированно вызывались деструкторы всех локальных объектов, необходимо использовать исключения или обычный механизм возврата управления и выхода из функции `main()`.

В стандарте C++11 семантика функции `quick_exit()` предусматривает не удаление объектов, а вызовы функций, зарегистрированных для вызова в функции `at_quick_exit()`, в порядке, обратном порядку их регистрации, и вызов функции `_Exit()`, прекращающей работу программы без удаления объектов и очистки³³. Это означает, что функции `quick_exit()` и `_Exit()` не очищают стандартные файловые буфера (стандартного вывода и ошибок).

В языке C++ обычным способом аварийного прекращения работы программы, представляющего собой неожиданный выход в противоположность ожидаемому выходу с выдачей сигналов об ошибках, является вызов функции `std::terminate()`, которая по умолчанию вызывается функцией `abort()`. Это происходит, например, если деструктор или функция, объявленная с ключевым словом `noexcept` (см. раздел 3.1.7), генерирует исключение.

5.8.3. Определения в заголовочном файле `<cstring>`

В табл. 5.26 приведены наиболее важные определения заголовочного файла `<cstring>`: низкоуровневые функции для установки, копирования и перемещения участков памяти. Одним из применений этих функций являются свойства символов (см. раздел 16.1.4).

Таблица 5.26. Определения в заголовочном файле `<cstring>`

Определение	Смысл
<code>memchr(const void* ptr, int c, size_t len)</code>	Находит символ <i>c</i> среди первых <i>len</i> байтов, на которые ссылается указатель <i>ptr</i>
<code>memcmp(const void* ptr1, const void* ptr2, size_t len)</code>	Сравнивает <i>len</i> байтов памяти, на которую ссылаются указатели <i>ptr1</i> и <i>ptr2</i>
<code>memcpy(void* toPtr, const void* fromPtr, size_t len)</code>	Копирует <i>len</i> байтов из памяти, на которую ссылается указатель <i>fromPtr</i> , в память, на которую ссылается указатель <i>Ptr</i>
<code>memmove(void* toPtr, const void* fromPtr, size_t len)</code>	Копирует <i>len</i> байтов из памяти, на которую ссылается указатель <i>fromPtr</i> , в память, на которую ссылается указатель <i>toPtr</i> (области могут перекрываться)
<code>memset(void* ptr, int c, size_t len)</code>	Присваивает символ <i>c</i> первым <i>len</i> байтам памяти, на которую ссылается указатель <i>ptr</i>

³³ Этот механизм позволяет предотвратить доступ отсоединенных потоков к глобальным/статическим объектам (см. раздел 18.2.1).